



Patrones de software

Implementación de patrones final

Docente: ELIECER MONTERO OJEDA

Ángel David Reyes Moreno

UNIDADES TECNOLÓGICAS DE SANTANDER
Ingeniería de Sistemas
Bucaramanga, 2026

PATRONES CREACIONALES

Patrón 1: Singleton

¿Qué problema resuelve? Garantiza que solo exista una única instancia de un objeto durante toda la vida de la aplicación. Útil para configuración y conexiones a bases de datos.

```
export class ConfigManager {  
  // 1. Variable estática que guarda LA única instancia  
  private static instance: ConfigManager | null = null;  
  
  // 2. Constructor PRIVADO → nadie puede hacer "new ConfigManager()"   
  private constructor() {  
    this.config = { /* Lee las variables de entorno */ }  
  }  
  
  // 3. Punto de acceso global → siempre devuelve la misma instancia  
  public static getInstance(): ConfigManager {  
    if (!ConfigManager.instance) {  
      ConfigManager.instance = new ConfigManager(); // solo se crea UNA vez  
    }  
    return ConfigManager.instance; // las demás veces, devuelve la existente  
  }  
}
```

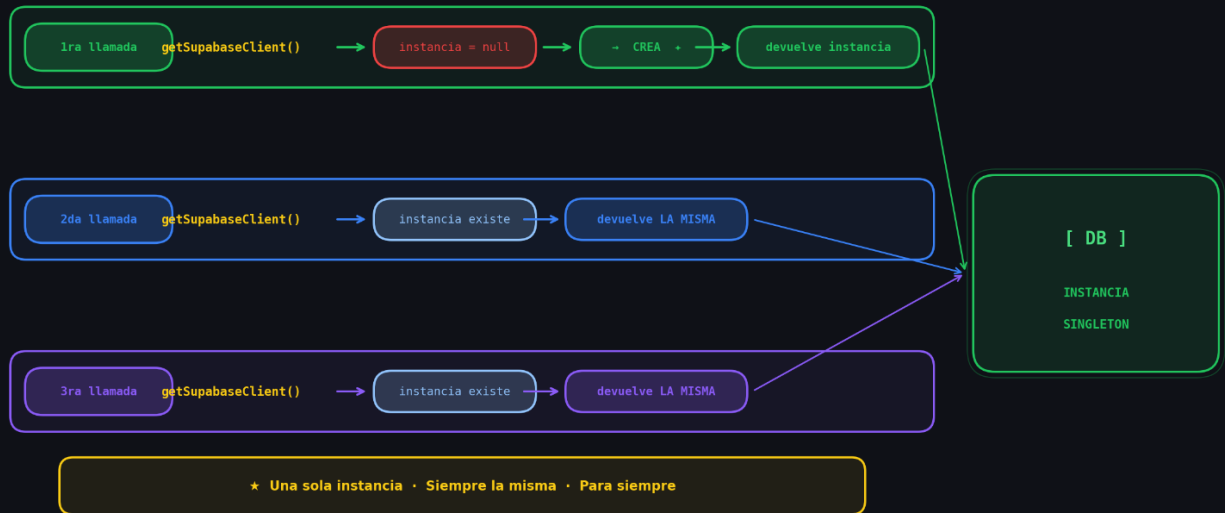
¿Dónde se usa? El cliente de Supabase lo llama al iniciarse:

```
let supabaseInstance: SupabaseClient | null = null;  
  
export function getSupabaseClient(): SupabaseClient {  
  if (supabaseInstance) return supabaseInstance; // ya existe → devuelve la misma  
  
  const config = ConfigManager.getInstance(); // + usa el otro Singleton  
  supabaseInstance = createClient(config.get("supabaseUrl"), ...);  
  return supabaseInstance;  
}
```

Diagrama Uml

PATRÓN SINGLETON

getSupabaseClient()



Patrón 2: Factory Method

¿Qué problema resuelve? Delegar la **creación de objetos** a subclases, sin que el código cliente sepa qué clase concreta se está creando. Perfecto para múltiples tipos de dispositivos IoT.

```
// Creator abstracto define el "molde"
abstract class DeviceAdapterCreator {
  abstract createAdapter(id: string, type: DeviceType): IDeviceAdapter; // ← Factory Method
}

// Cada subclase decide QUÉ adaptador concreto crear
class SimulatedAdapterCreator extends DeviceAdapterCreator {
  createAdapter(id: string, type: DeviceType): IDeviceAdapter {
    return new SimulatedDeviceAdapter(id, type); // concreta
  }
}

// Punto de entrada público — elige el Creator correcto
export class DeviceAdapterFactory {
  static async create(params): Promise<IDeviceAdapter> {
    switch (params.protocol) {
      case "MQTT":      return new MQTTAdapterCreator().getAndConnect(...);
      case "REST":      return new RESTAdapterCreator().getAndConnect(...);
      case "SIMULATED": return new SimulatedAdapterCreator().getAndConnect(...);
    }
  }
}
```

¿Dónde se usa en la UI?

La UI solo llama a la fábrica. De momento se está instanciando un protocolo de simulación si en un futuro se implementa un protocolo en tiempo real solo hay que crear ese protocolo ejemplo: ZIGBEE, solo ZigbeeDeviceAdapter y ZigbeeAdapterCreator — el código de la UI no cambia para nada seguirá llamando a la fábrica.

```
// La página NO sabe qué adaptador concreto se crea
const adapter = await DeviceAdapterFactory.create({
  deviceId: "dev-001",
  deviceType: "SOLAR_PANEL",
  protocol: "SIMULATED", // ← aquí solo cambia el protocolo
});
const reading = await adapter.readData(); // misma interfaz siempre
```

Diagrama Uml

FACTORY METHOD — Device Adapter

Patron de disenio creacional

«interface»

IDeviceAdapter

connect() · readData() · disconnect()

Contrato de comunicacion con dispositivos

implementaciones concretas

MQTTDeviceAdapter

connect()
(fisica real)

RESTDeviceAdapter

connect()
(API HTTP)

SimulatedDeviceAdapter

connect()^{crea / usa}
(datos sinteticos)

«abstract»

DeviceAdapterCreator

createAdapter() — Factory Method

subclases deben implementar createAdapter()

creators concretos

MQTTAdapterCreator

createAdapter()
retorna instancia concreta

RESTAdapterCreator

createAdapter()
retorna instancia concreta

SimulatedAdapterCreator

createAdapter()
retorna instancia concreta

★ Cada Creator sabe cual Adapter crear — el cliente solo habla con IDeviceAdapter

Patrón 3: Abstract Factory

¿Qué problema resuelve?

En la plataforma tenemos **4 fuentes de energía** (Solar, Eólica, Batería, Red). Cada una necesita tres componentes que **deben ser coherentes entre sí**:

- Un lector de producción
- Un calculador de precios
- Un modelo de predicción

Sin el patrón, la UI tendría que saber qué clase concreta instanciar en cada caso, generando código espagueti

¿Dónde se usa?

Se define las tres faces del producto

```
interface IEnergyProducer { getCurrentReading(): Promise<EnergyReading>; }
interface IPriceEstimator { getCurrentQuote(): Promise<PriceQuote>; }
interface IForecastModel { getDailyForecast(): Promise<DailyForecast>; }
```

Se define la Abstract Factory (el contrato de la fábrica)

```
interface IEnergySourceFactory {
  createProducer(): IEnergyProducer; // + crea producto A
  createPriceEstimator(): IPriceEstimator; // + crea producto B
  createForecastModel(): IForecastModel; // + crea producto C
}
```

Se implementan las 4 fábricas concretas, una por familia

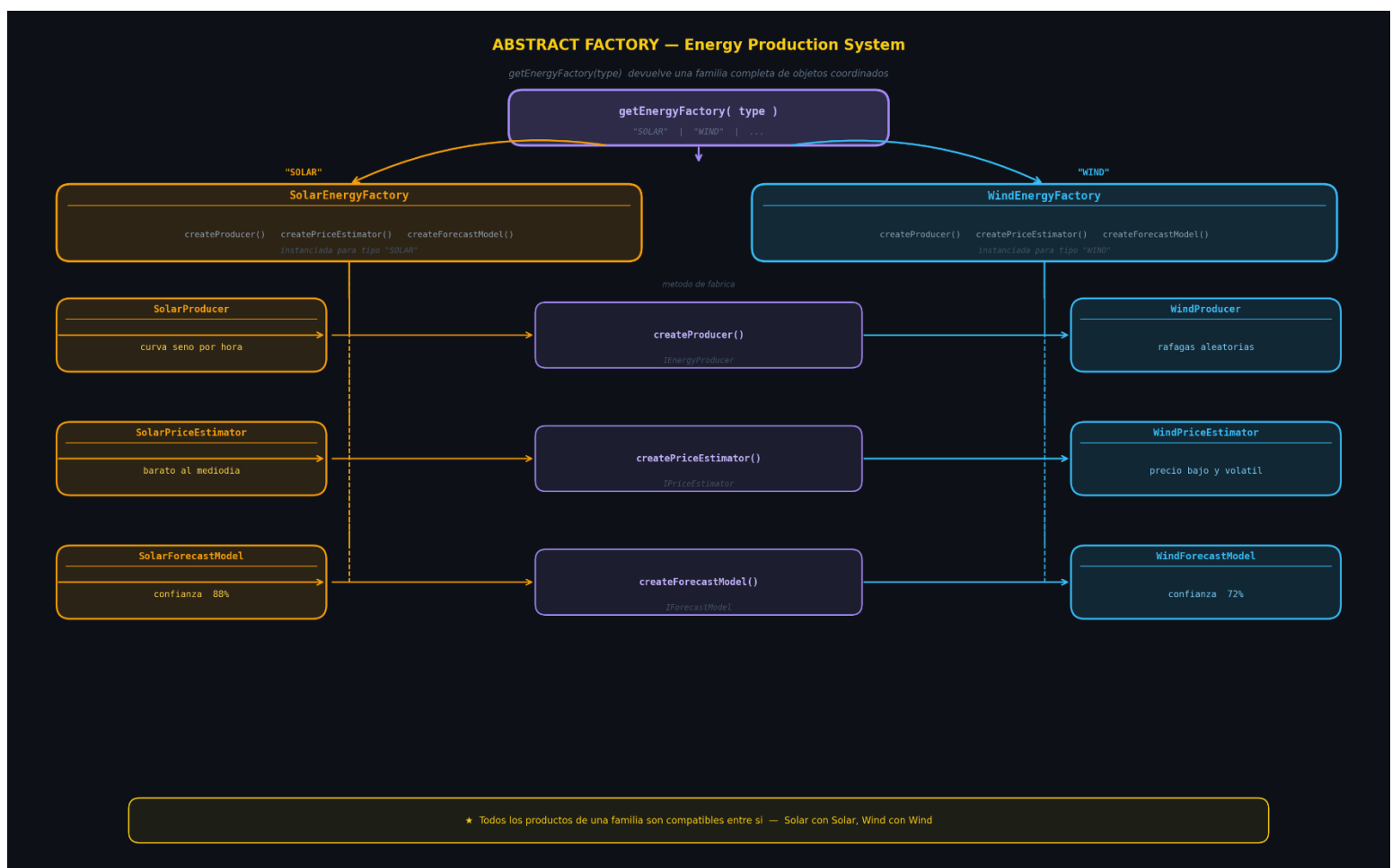
```
class SolarEnergyFactory implements IEnergySourceFactory {
  createProducer() { return new SolarProducer(); } // curva seno solar
  createPriceEstimator() { return new SolarPriceEstimator(); } // más barato al mediodía
  createForecastModel() { return new SolarForecastModel(); } // confianza 88%
}

class WindEnergyFactory ... // variación tipo ráfaga, confianza 72%
class BatteryEnergyFactory ... // carga/descarga, confianza 95%
class GridEnergyFactory ... // ilimitada, la más cara, confianza 99%
```

Por ultimo el selector de las cuatro clases de fabrica

```
export function getEnergyFactory(source: EnergySource): IEnergySourceFactory {  
  switch (source) {  
    case "SOLAR": return new SolarEnergyFactory();  
    case "WIND": return new WindEnergyFactory();  
    case "BATTERY": return new BatteryEnergyFactory();  
    case "GRID": return new GridEnergyFactory();  
  }  
}
```

Diagrama Uml



Patrón 4: Builder

¿Qué problema resuelve?

Una TradeOrder tiene muchos campos opcionales. Sin Builder, el constructor se convierte en un desastre:

```
// ❌ Sin Builder - imposible saber qué significa cada argumento  
new TradeOrder("SELL", 12.5, 0.124, "SOLAR", "FIXED", "URGENT", 30, true, false, "Excedente")
```

Con Builder se construye la orden **paso a paso**, solo con lo que se necesita :

```
new TradeOrderBuilder()  
  .ofType("SELL")  
  .withAmount(12.5)  
  .atPrice(0.124)  
  .fromSource("SOLAR")  
  .withPriority("URGENT")  
  .build()
```

¿Dónde se usa?

El patrón tiene **3 participantes** hasta el momento en el proyecto

1. El Producto — TradeOrder (interfaz)

El objeto complejo que se construye al final:

```
interface TradeOrder {  
  id: string;          type: OrderType;      amountKwh: number;  
  pricePerKwh: number; energySource: EnergySource;  
  pricingMode: PricingMode; priority: OrderPriority;  
  expiresAt: Date | null; note: string | null;  
  conditions: TradeOrderConditions;  
  maxSlippagePercent: number; totalValueUsd: number;  
  status: OrderStatus;  
}
```

2. El Builder — TradeOrderBuilder

Acumula el estado internamente y expone métodos encadenables:

```
class TradeOrderBuilder {
  // campos privados con valores por defecto seguros
  private _type: OrderType | null = null;
  private _amountKwh: number | null = null;
  private _priority: OrderPriority = "NORMAL"; // default
  private _conditions: TradeOrderConditions = {};
  // ...

  ofType(type)      { this._type = type;      return this; }
  withAmount(kwh)   { this._amountKwh = kwh;   return this; }
  atPrice(usd)      { this._pricePerKwh = usd; return this; }
  fromSource(src)   { this._energySource = src; return this; }
  expiresInMinutes(min) { /* calcula Date */    return this; }
  requireGreenCertified() { this._conditions.requireGreenCertified = true; return this; }

  build(): TradeOrder {
    // Valida campos requeridos
    if (!this._type) throw new TradeOrderBuildError("El tipo es requerido");
    if (this._pricePerKwh <= 0) throw new TradeOrderBuildError("El precio debe ser > 0");
    // Calcula campos derivados y retorna el producto final
    return { id: `ORD-...`, totalValueUsd: kwh * price, status: "DRAFT", ... };
  }

  reset() { /* Limpia todo para reutilizar */ return this; }
}
```

3. El Director — TradeOrderDirector

Encapsula **recetas predefinidas** usando el Builder. Sabe qué pasos llamar y en qué orden:

```
class TradeOrderDirector {
    constructor(private builder: TradeOrderBuilder) {}

    buildUrgentSolarSell(kwh, price): TradeOrder {
        return this.builder.reset()
            .ofType("SELL").withAmount(kwh).atPrice(price)
            .fromSource("SOLAR").withPricingMode("DYNAMIC")
            .withPriority("URGENT").expiresInMinutes(30)
            .withMaxSlippage(5).allowPartialFill()
            .build();
    }

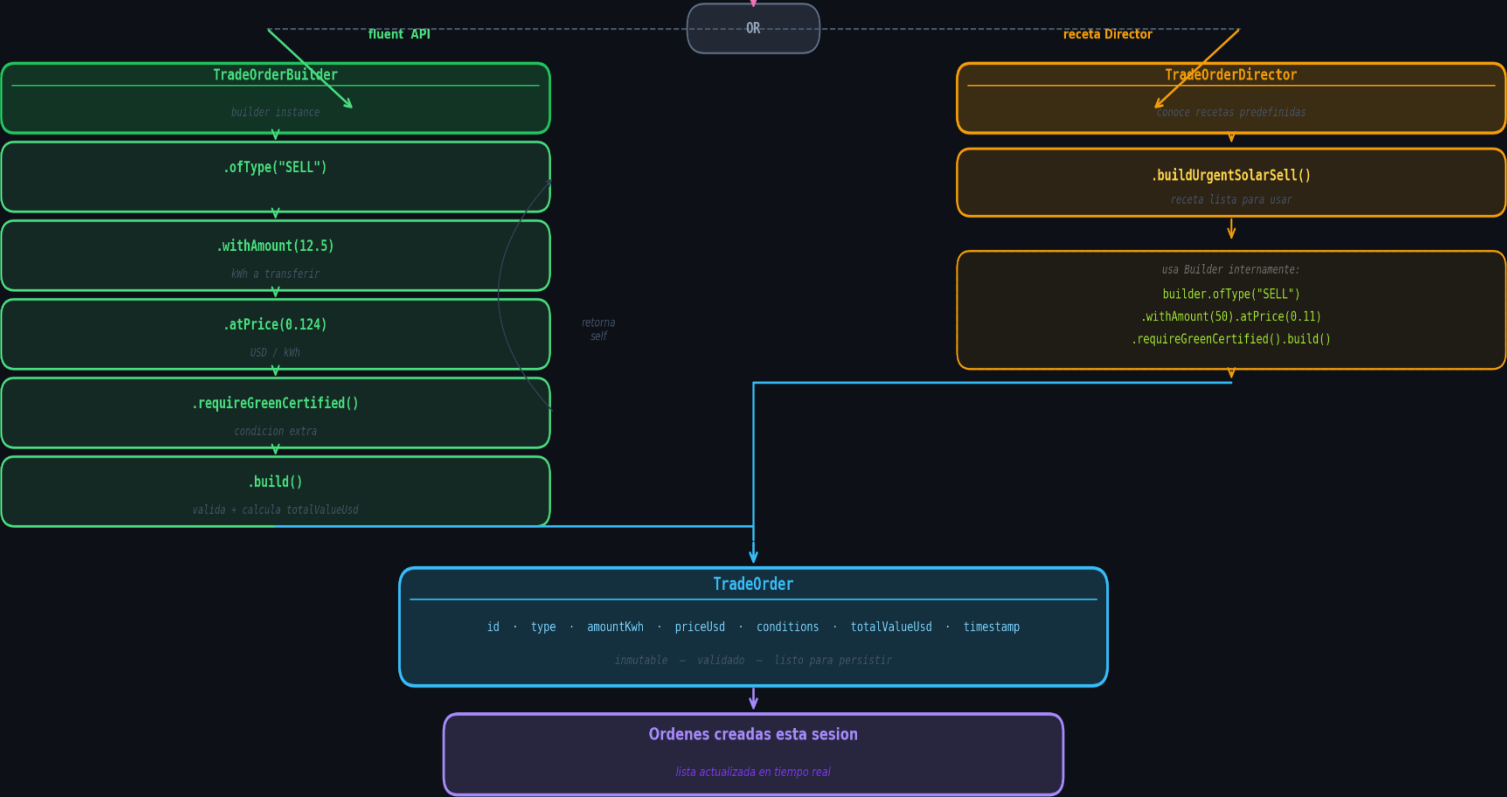
    buildScheduledGreenBuy(kwh, maxPrice): TradeOrder { ... }
    buildPeakBatteryDischarge(kwh, price): TradeOrder { ... }
}
```

Diagrama Uml

PATRON BUILDER — TradeOrder Construction

Dos rutas para construir la misma orden — fluent API o Director con receta

Usuario llena el formulario / hace clic en receta
campos: tipo, cantidad, precio, condiciones



LEYENDA

- Usuario / entrada
- Builder (fluent API)
- Director (receta)
- Producto final: TradeOrder
- UI / vista

★ El Director es opcional — simplifica la creacion de objetos complejos con configuraciones frecuentes

Patrón 5: Prototype

¿Qué problema resuelve?

Un trader vende 10 kWh solares cada mañana. Sin Prototype, lo reconstruye con el Builder desde cero todos los días. Con Prototype, guarda esa orden como plantilla y la clona en segundos el clon es 100% independiente, tiene nuevo ID y puede pisar campos.

¿Dónde se usa?

```
interface IOrderPrototype {  
  clone(): OrderTemplate;  
  getOrder(): TradeOrder;  
}
```

```
class OrderTemplate implements IOrderPrototype {  
  private readonly _order: TradeOrder; // orden congelada (immutable)  
  
  clone(overrides: OrderOverrides = {}): OrderTemplate {  
    this.usageCount++;  
  
    // Reconstruye con el Builder → nuevo ID, validación garantizada  
    const builder = new TradeOrderBuilder()  
      .ofType(this._order.type)  
      .withAmount(overrides.amountKwh ?? this._order.amountKwh) // ← override opcional  
      .atPrice(overrides.pricePerKwh ?? this._order.pricePerKwh)  
      .fromSource(this._order.energySource)  
      // ... copia todas las condiciones del original  
      .withNote(`Clon de: ${this.name}] ${this._order.note}`);  
  
    return new OrderTemplate(builder.build(), `${this.name} (copia)`);  
  }  
}
```

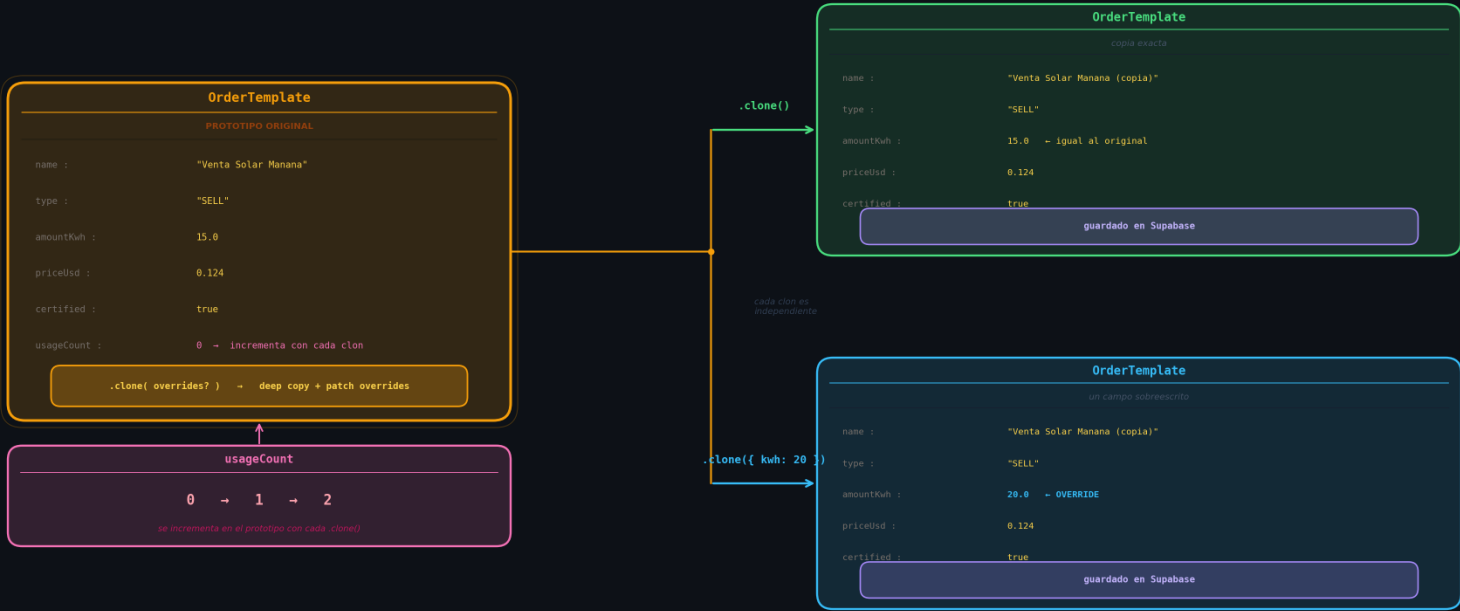
Registra las plantillas y las manda a la base de datos de supabase

```
class OrderTemplateRegistry {  
  // 3 templates pre-cargados: Venta Solar Mañana, Carga Batería Noche, Compra Verde  
  
  cloneFrom("Venta Solar Mañana", { amountKwh: 15 })  
  // → nueva TradeOrder: 15 kWh (override), precio/fuente/condiciones del original  
}
```


Diagrama Uml

PATRON PROTOTYPE — OrderTemplate Cloning

Crea nuevas ordenes copiando una plantilla existente — sin reescribir todos los campos



★ El prototipo nunca se modifica — cada `.clone()` produce un objeto independiente listo para persistir

PATRONES ESTRUCTURALES

Patrón 1: Adapter

¿Qué problema resuelve?

el Adapter traduce la interfaz incompatible de cada proveedor externo a la interfaz estándar IPriceEstimator. El resto del sistema (páginas, servicios) solo conoce IPriceEstimator y no sabe nada del proveedor real.

¿Dónde se usa?

1.Target: la interfaz que el sistema usa internamente:

```
interface IPriceEstimator {  
  getCurrentQuote(): Promise<PriceQuote>; // → siempre USD/kWh  
}
```

2 Adaptees: las interfaces incompatibles de cada proveedor:

```
interface IOmieMarketApi {  
  fetchDayAheadPrice(fecha: string): Promise<{  
    precio_mwh: number; // ← €/MWh (no USD/kWh)  
    tecnologia: "SOLAR" | "EOLICA" | "HIDRO" | "TERMICO"; // ← español  
    variacion_pct: number;  
  }>;  
}
```

3 Los Adapters: traducen el formato externo al interno:

```

class OmieAdapter implements IPriceEstimator { // + implementa Target
  constructor(private omieApi: IOmieMarketApi) {} // + envuelve al Adaptee

  async getCurrentQuote(): Promise<PriceQuote> {
    const raw = await this.omieApi.fetchDayAheadPrice(today);

    // TRADUCCIÓN: €/MWh → USD/kWh
    const priceUsdKwh = raw.precio_mwh * 1.08 * 0.001;

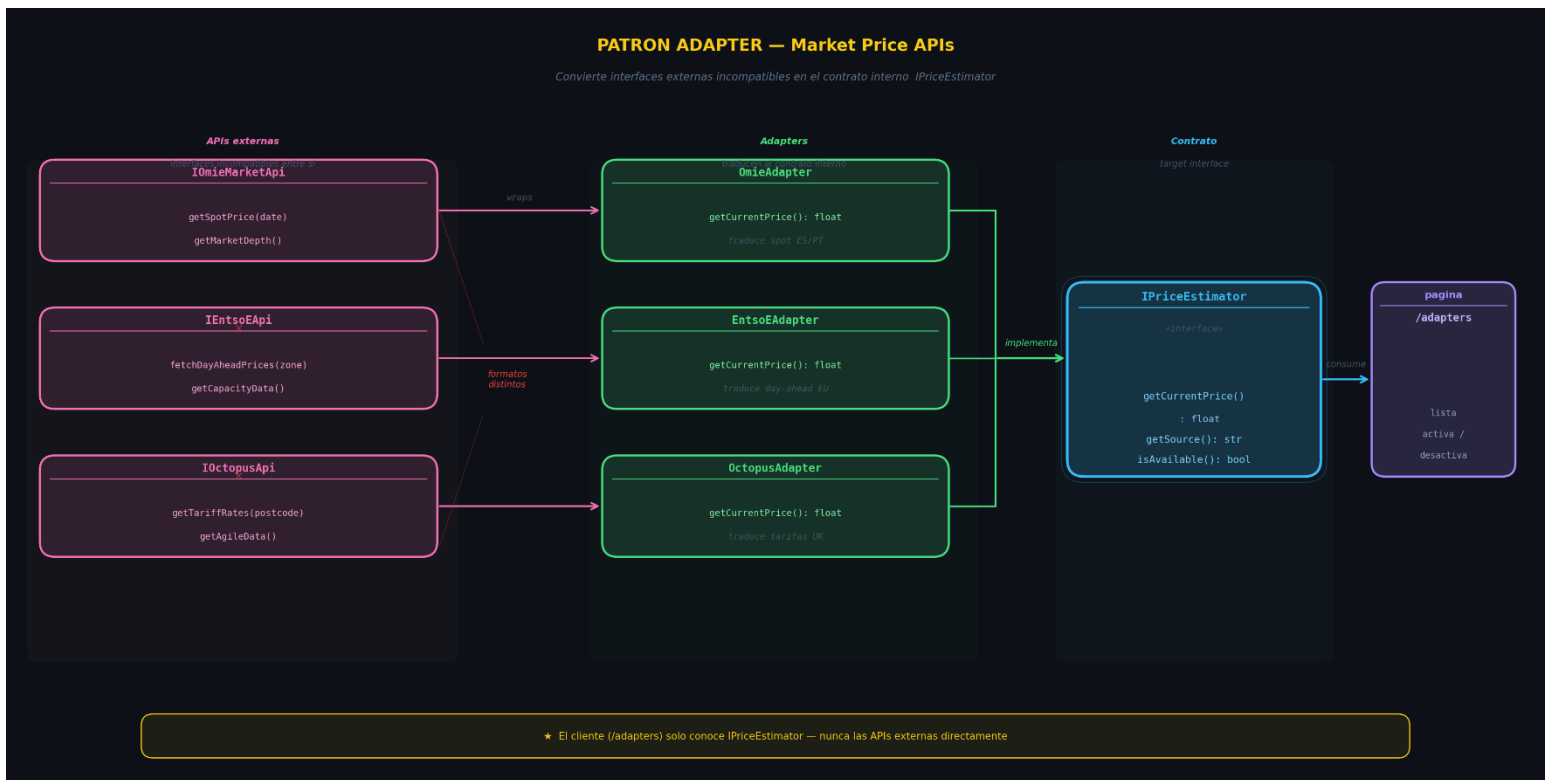
    // TRADUCCIÓN: "EOLICA" → "WIND"
    const source = { SOLAR:"SOLAR", EOLICA:"WIND", ... }[raw.tecnologia];

    // TRADUCCIÓN: variacion_pct → "UP"/"DOWN"/"STABLE"
    const trend = raw.variacion_pct > 1 ? "UP" : ...

    return { source, pricePerKwh: priceUsdKwh, trend, currency: "USD", ... };
  }
}

```

Diagrama Uml



Patrón 2: Bridge

¿Qué problema resuelve?

el sistema genera notificaciones de 3 tipos distintos (precio, orden, dispositivo) y necesita entregarlas por múltiples canales (consola, in-app, email, webhook). Sin Bridge, habría $3 \times 4 = 12$ clases. Con Bridge: $3 + 4 = 7$ clases

¿Dónde se usa?

IMPLEMENTOR — la jerarquía de cómo se entrega:

```
interface INotificationChannel {  
    deliver(notification: Notification): Promise<void>;  
}  
  
class ConsoleChannel implements INotificationChannel { ... } //  
imprime en terminal  
class InAppChannel implements INotificationChannel { ... } //  
guarda en memoria → UI  
class EmailChannel implements INotificationChannel { ... } //  
simula SendGrid  
class WebhookChannel implements INotificationChannel { ... } //  
simula POST HTTP
```

ABSTRACTION — la clase base que compone el canal (no hereda):

```
abstract class Notifier {  
    constructor(protected channel: INotificationChannel) {} // ← el  
    "bridge"  
  
    setChannel(channel: INotificationChannel) { // cambia canal en  
        runtime  
        this.channel = channel;  
    }  
}
```

REFINED ABSTRACTIONS — la jerarquía de qué se notifica:

```

class PriceAlertNotifier extends Notifier {
  async alertPriceSpike(source, price, threshold) {
    // formatea el mensaje específico de precio...
    await this.channel.deliver({ title: "Precio alto", level:
"CRITICAL", ... });
  }
}

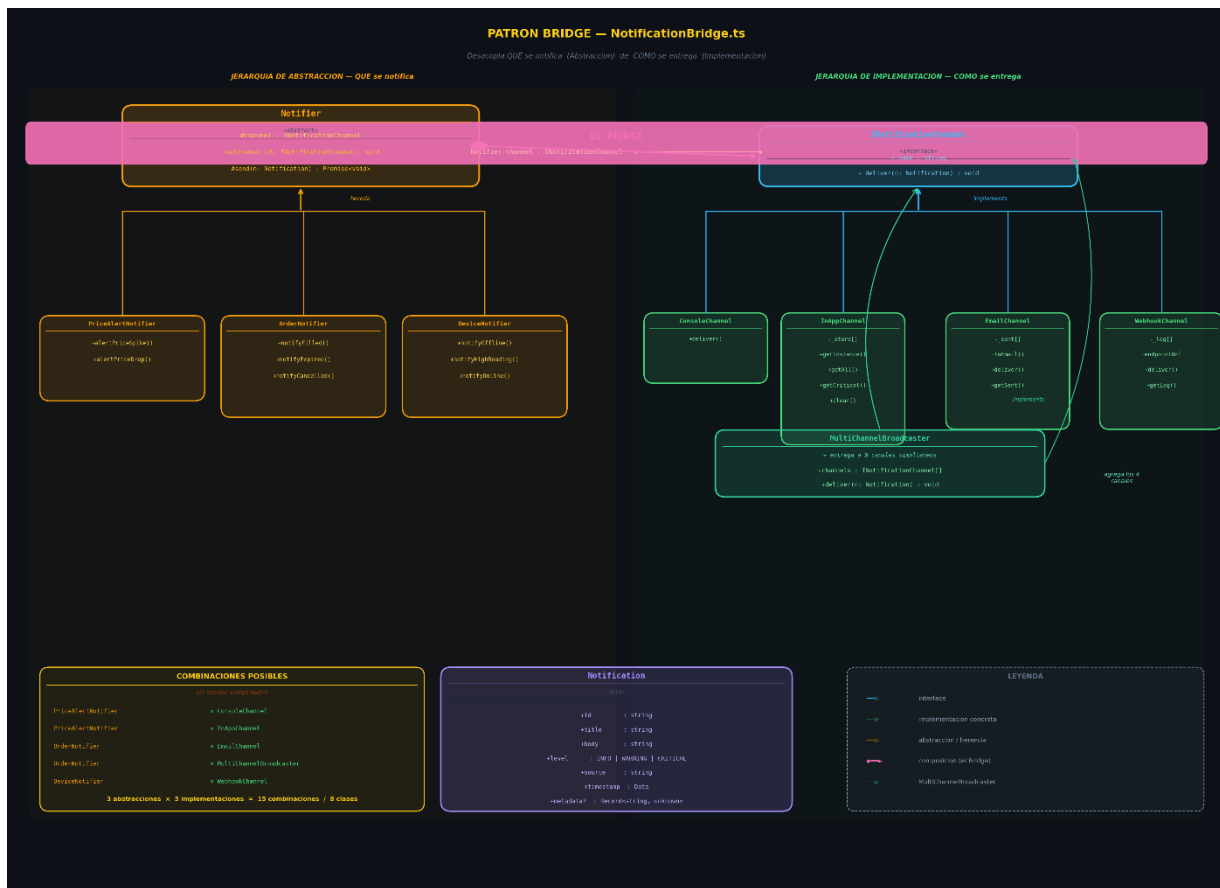
class OrderNotifier extends Notifier {
  async notifyOrderFilled(orderId, kwh, price) { ... }
  async notifyOrderExpired(orderId, reason) { ... }
}

class DeviceNotifier extends Notifier {
  async notifyDeviceOffline(deviceId, name) { ... }
}

```

El "Bridge" es la referencia **protected channel** en la clase base. Al cambiar el canal, todo el comportamiento de entrega cambia sin tocar ninguna de las 3 clases de notificación:

Diagrama Uml



Patrón 3: Decorator

¿Qué problema resuelve?

El OmieAdapter (del patrón Adapter) obtiene precios, pero falta:

- ¿Cómo evitar llamar la API 60 veces por minuto? → **Caché**
- ¿Cómo saber si los precios subieron de golpe? → **Alertas**
- ¿Cómo registrar cada consulta para análisis? → **Logging**
- ¿Qué pasa si la red falla? → **Reintentos**

Sin Decorator: habría que meter todo eso dentro del OmieAdapter, mezclando responsabilidades. Con Decorator: **cada comportamiento es una capa independiente y apilable.**

¿Dónde se usa?

COMPONENT — la interfaz del patrón Adapter:

```
interface IPriceEstimator {  
  getCurrentQuote(): Promise<PriceQuote>;  
}
```

DECORATOR BASE — envuelve cualquier IPriceEstimator:

```
abstract class PriceEstimatorDecorator implements IPriceEstimator {  
  constructor(protected readonly wrapped: IPriceEstimator) {}  
  
  getCurrentQuote(): Promise<PriceQuote> {  
    return this.wrapped.getCurrentQuote(); // delega por defecto  
  }  
}
```

4 DECORADORES CONCRETOS — cada uno añade una sola responsabilidad:

```

class CachingDecorator extends PriceEstimatorDecorator {
  async getCurrentQuote() {
    if (cache valida) return this._cached; // corto-circuito
    const quote = await this.wrapped.getCurrentQuote(); // delega y cachea
    this._cached = quote;
    return quote;
  }
  getStats() → { hits, misses, ratio }
}

class LoggingDecorator extends PriceEstimatorDecorator {
  async getCurrentQuote() {
    const start = Date.now();
    const quote = await this.wrapped.getCurrentQuote();
    this._history.unshift({ price, durationMs, fromCache, ... });
    return quote;
  }
}

class AlertingDecorator extends PriceEstimatorDecorator {
  async getCurrentQuote() {
    const quote = await this.wrapped.getCurrentQuote();
    if (quote.pricePerKwh > this.spikeThreshold) {
      // USA EL BRIDGE para entregar la alerta + integración entre patrones
      new PriceAlertNotifier(InAppChannel.getInstance()).alertPriceSpike(...);
    }
    return quote;
  }
}

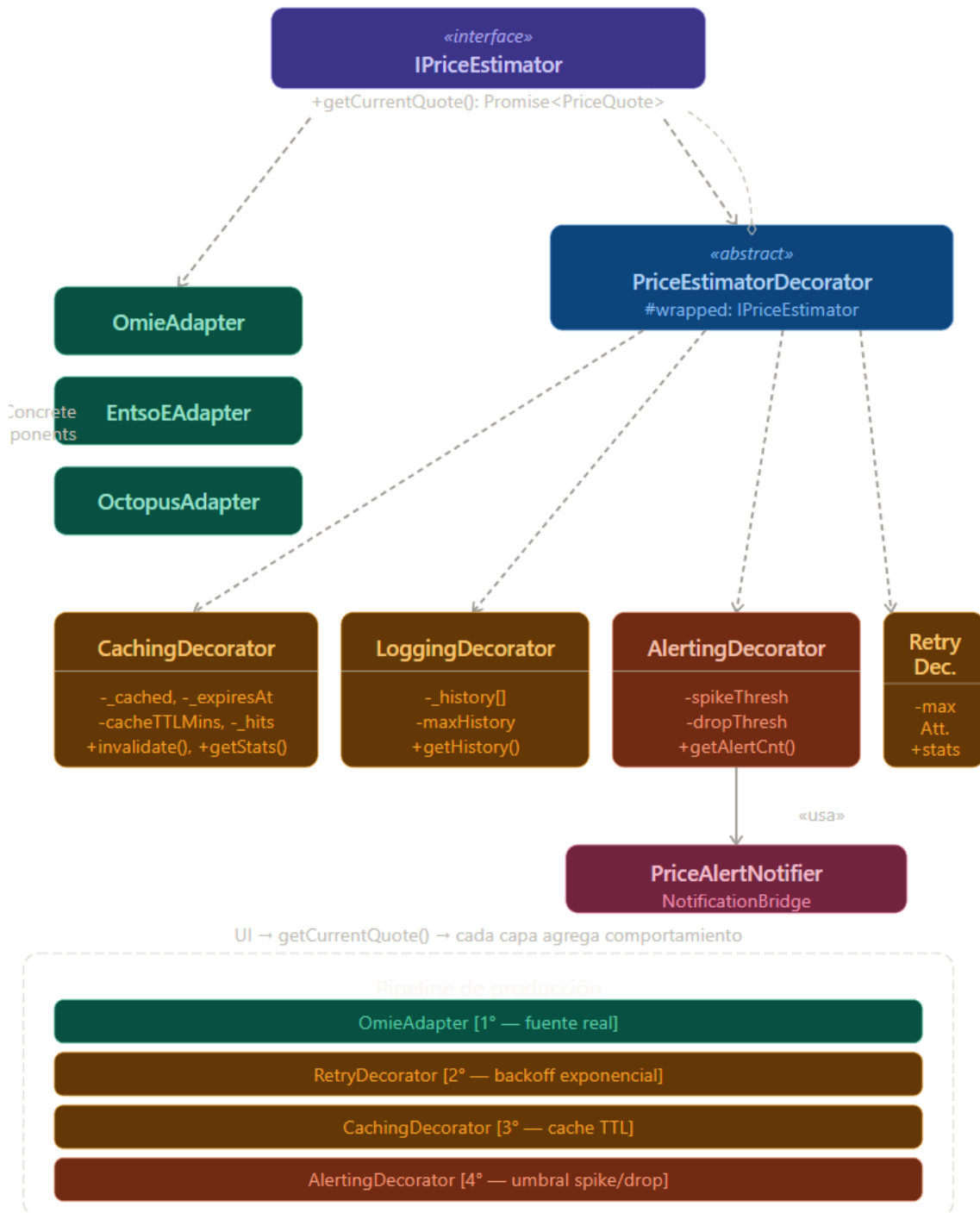
class RetryDecorator extends PriceEstimatorDecorator {
  async getCurrentQuote() {
    for (let i = 1; i <= maxAttempts; i++) {
      try { return await this.wrapped.getCurrentQuote(); }
      catch { await sleep(baseDelay * 2^i); } // backoff exponencial
    }
  }
}

```

Pipeline completo — los decoradores se apilan de adentro hacia afuera:

```
const pipeline = buildPricePipeline(new OmieAdapter(api), {  
  cacheTTLMinutes: 1,  
  spikeThreshold: 0.18,  
  maxRetries: 3,  
});  
  
// Capa más externa → Logging  
//   → Alerting  
//     → Caching  
//       → Retry  
//         → OmieAdapter (capa más interna)
```


Diagrama Uml



Patrón 4: Facade

¿Qué problema resuelve?

Antes de la Facade, para que una página cree y publique una orden necesitaba conocer y coordinar 5 subsistemas por separado.

```
// ❌ Sin Facade - el cliente debe conocer TODOS los subsistemas:  
const builder = new TradeOrderBuilder(); // subsistema 1  
const order = builder.ofType("SELL")...build();  
const db = getSupabaseAdmin(); // subsistema 2  
await db.from("energy_orders").insert({...});  
const notifier = new OrderNotifier(InAppChannel.getInstance()); // subsistema 3  
await notifier.notifyOrderFilled(order.id, ...);
```

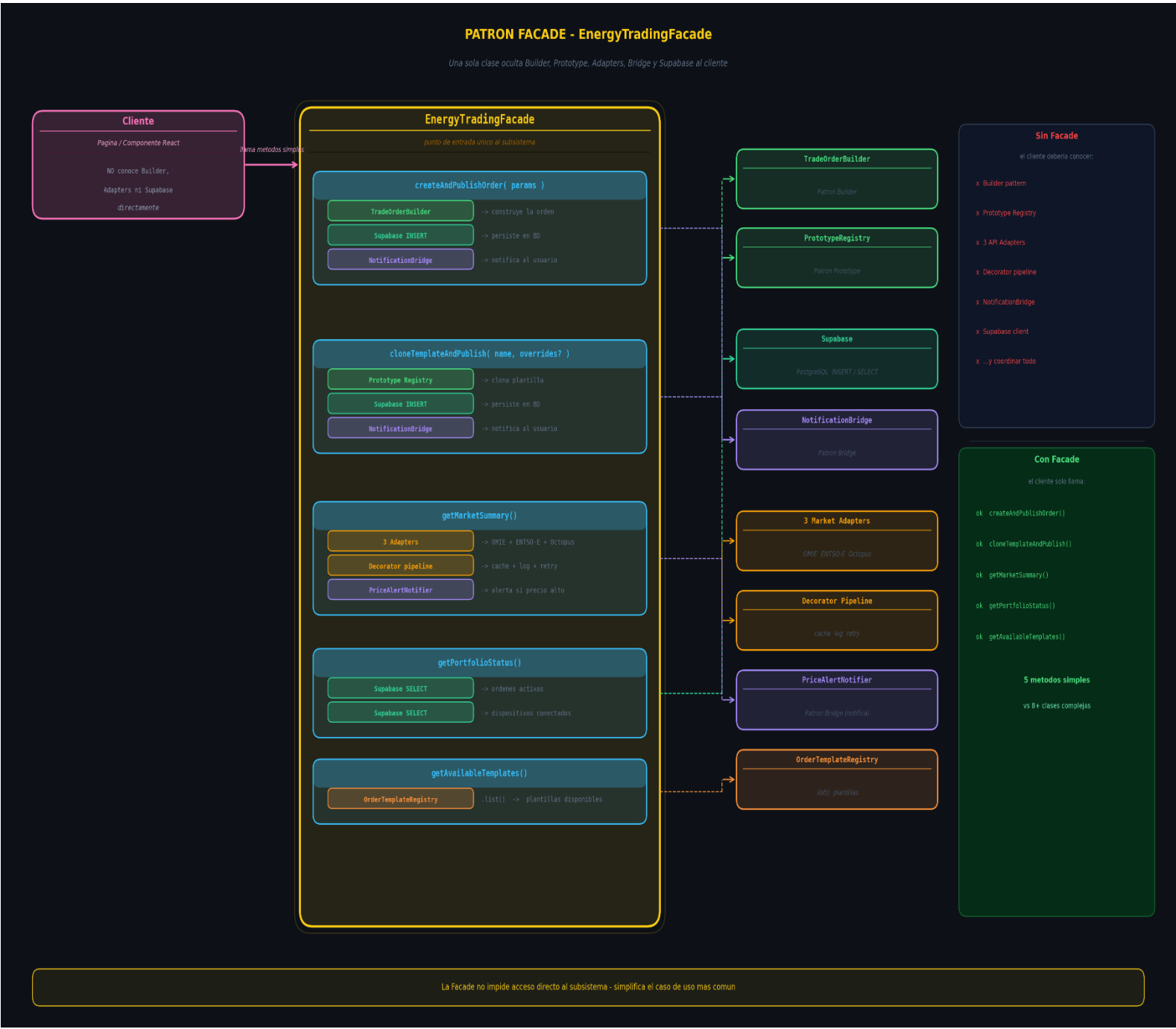
EnergyTradingFacade expone operaciones de alto nivel que internamente coordinan todos los subsistemas. El cliente solo llama a la Facade y obtiene el resultado.

¿Dónde se usa?

La Facade expone 5 operaciones de alto nivel, cada una orquestando subsistemas distintos:

```
async createAndPublishOrder(params: CreateOrderParams): Promise<FacadeResult<string>> {  
  // Subsistema 1 - Builder  
  const order = new TradeOrderBuilder().ofType(params.type)...build();  
  
  // Subsistema 2 - Supabase  
  const { error } = await getSupabaseAdmin().from("energy_orders").insert({...});  
  
  // Subsistema 3 - Bridge de notificaciones  
  await new OrderNotifier(InAppChannel.getInstance()).notifyOrderFilled(...);  
  
  return { ok: true, data: order.id };  
}
```

Diagrama Uml



Patrón 5: composite

¿Qué problema resuelve?

Una red de energía tiene dispositivos individuales y grupos de dispositivos. Sin Composite, el código tiene que preguntar constantemente "¿es un dispositivo o es un grupo?"

```
// ❌ Sin Composite - Lógica diferente para cada caso:
function calcularTotal(nodo: unknown): number {
  if (nodo instanceof EnergyDevice) {
    return nodo.energyKwh; // caso hoja
  } else if (nodo instanceof EnergyGroup) {
    return nodo.children.reduce((sum, c) => sum + calcularTotal(c), 0); // caso grupo
  }
  return 0;
}
```

Con Composite, ambos implementan IEnergyNode y el cliente llama siempre lo mismo

```
// ✅ Con Composite - el cliente no sabe si es hoja o grupo:
unPanel.getTotalKwh() // → 42.5 kWh
unaGranja.getTotalKwh() // → 91.8 kWh (suma recursiva de 3 paneles)
elPortfolio.getTotalKwh() // → 593.8 kWh (suma de toda la jerarquía)
```

¿Dónde se usa?

Component — la interfaz común

```
interface IEnergyNode {
  getTotalKwh(): number; // energía total
  getPeakPowerKw(): number; // potencia pico
  getStatus(): NodeStatus; // ONLINE | PARTIAL | OFFLINE
  getChildCount(): number; // 0 si Leaf, N si Composite
  describe(): string; // árbol de texto
}
```

Leaf — un dispositivo físico individual:

```
class EnergyDevice implements IEnergyNode {
  getTotalKwh() → this.snap.energyKwh // su propio valor
  getPeakPowerKw() → this.snap.peakPowerKw // su propio valor
  getChildCount() → 0 // siempre cero
}
```

Composite — un grupo que contiene otros nodos:

```

class EnergyGroup implements IEnergyNode {
    private _children: IEnergyNode[];    // puede ser Leaf u otro EnergyGroup

    // RECURSIVIDAD - delega a cada hijo, sin saber si es Leaf o Group
    getTotalKwh()    → this._children.reduce((sum, c) => sum + c.getTotalKwh(), 0)
    getPeakPowerKw() → this._children.reduce((sum, c) => sum + c.getPeakPowerKw(), 0)

    // ONLINE si todos online; PARTIAL si alguno; OFFLINE si ninguno
    getStatus() → allOnline ? "ONLINE" : anyOnline ? "PARTIAL" : "OFFLINE"

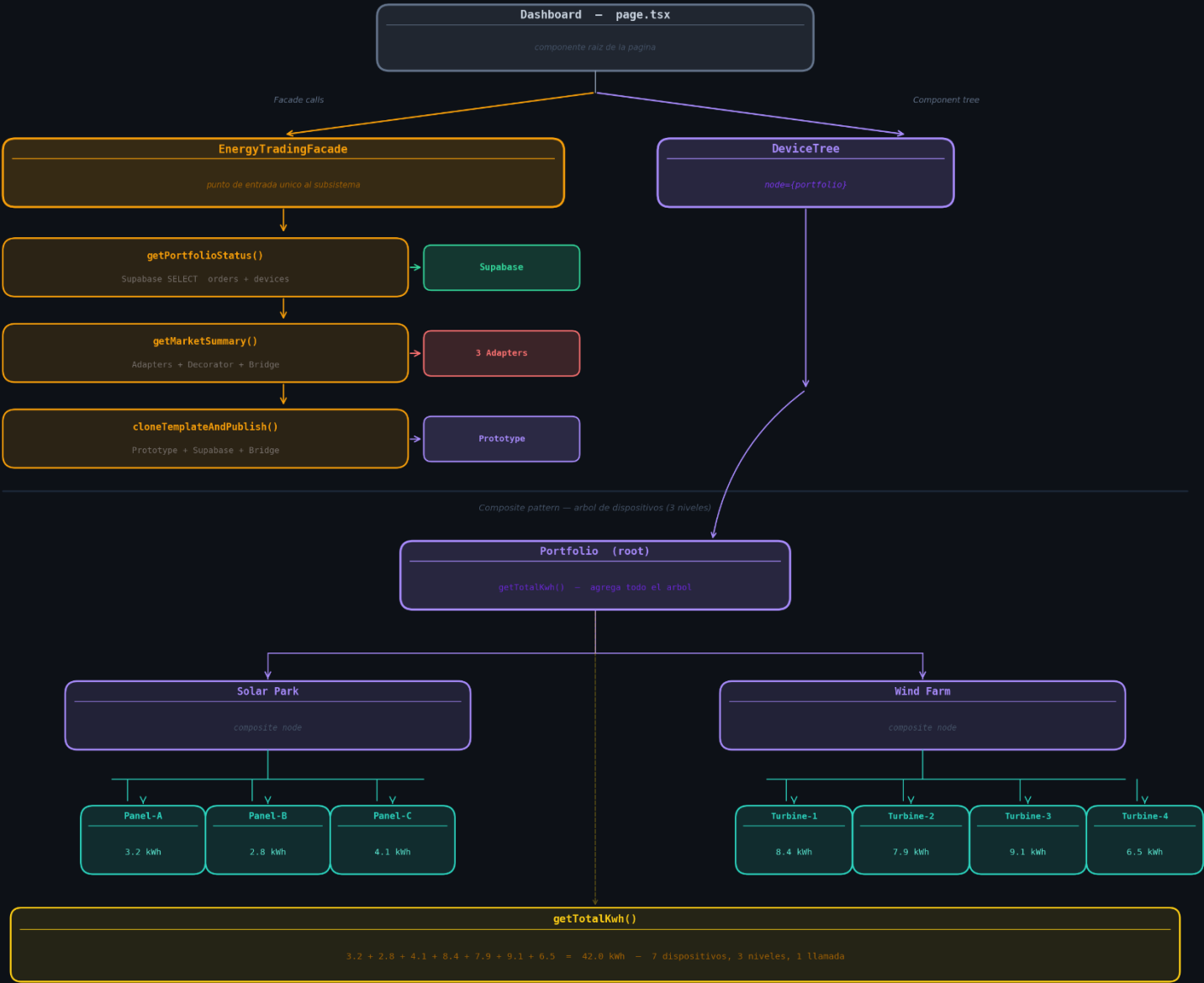
    // Cuenta todos los nodos en el subárbol (recursivo)
    getChildCount() → children.reduce((sum, c) => sum + 1 + c.getChildCount(), 0)
}

```

Diagrama Uml

FACADE + COMPOSITE — Dashboard Architecture

page.tsx coordina Facade para operaciones y Composite para el arbol de dispositivos



CLAVE DEL DIAGRAMA

- page.tsx / Dashboard**
componente React raiz, orquesta todo
- Supabase**
PostgreSQL — INSERT ordenes / SELECT portfolio
- Composite (Portfolio)**
arbol de dispositivos — leaf y branch misma interfaz
- EnergyTradingFacade**
oculta complejidad: Builder, Adapters, Prototype, Bridge
- 3 Adapters + Decorator**
OMIE, ENT50-E, Octopus con cache, log y retry
- Leaf devices (7 total)**
Panel-A/B/C, Turbine-1/2/3/4 — cada uno retorna kWh

Patrón 6: Flyweight

¿Qué problema resuelve?

Una red IoT con 50 dispositivos genera ~3000 lecturas/minuto. Sin Flyweight, cada objeto lectura almacena los mismos 5 campos repetidos en memoria:

```
Lectura #1 → { deviceId:"D1", timestamp:..., value:12.5, sensorType:"POWER", unit:"kW",  
label:"Potencia", color:"#f59e0b", range:{0-50} }  
Lectura #2 → { deviceId:"D1", timestamp:..., value:11.8, sensorType:"POWER", unit:"kW",  
label:"Potencia", color:"#f59e0b", range:{0-50} }  
Lectura #3 → { deviceId:"D2", timestamp:..., value:13.1, sensorType:"POWER", unit:"kW",  
label:"Potencia", color:"#f59e0b", range:{0-50} }  
...10,000 lecturas con los mismos 5 campos copiados ❌
```

Con Flyweight: esos 5 campos se almacenan una sola vez, compartidos:

```
Lectura #1 → { deviceId:"D1", timestamp:..., value:12.5, flyweight → ▶ }  
Lectura #2 → { deviceId:"D1", timestamp:..., value:11.8, flyweight → ▶ }  
Lectura #3 → { deviceId:"D2", timestamp:..., value:13.1, flyweight → ▶ }  
▶ { unit:"kW",  
label:"Potencia", color:"#f59e0b", range:{0-50} }  
...10,000 lecturas compartiendo 1 sola instancia ✅
```

¿Dónde se usa?

Estado intrínseco (compartido, inmutable) — `SensorTypeFlyweight`

```
// 1 sola instancia por tipo de sensor para toda la app  
interface SensorTypeFlyweight {  
    readonly sensorType: SensorType; // "POWER"  
    readonly unit: string; // "kW"  
    readonly displayLabel: string; // "Potencia"  
    readonly colorHex: string; // "#f59e0b"  
    readonly normalRange: { min, max }; // { min:0, max:50 }  
    format(value): string; // "12.50 kW"  
    classify(value): "NORMAL"|"HIGH"|"CRITICAL";  
}
```

Fábrica / Pool — `ReadingFlyweightFactory`:

```

class ReadingFlyweightFactory {
    private static readonly _pool = new Map<SensorType, SensorTypeFlyweight>();
    // Pre-carga los 8 tipos al arrancar el módulo

    static get(sensorType: SensorType): SensorTypeFlyweight {
        // Si ya existe → devuelve el mismo objeto (cache hit)
        // Si no → crea uno nuevo y lo guarda en el pool
    }

    static getStats() → { poolSize:8, totalRequests, cacheHits, hitRatio }
}

```

Estado extrínseco (único por lectura) — IoTReading:

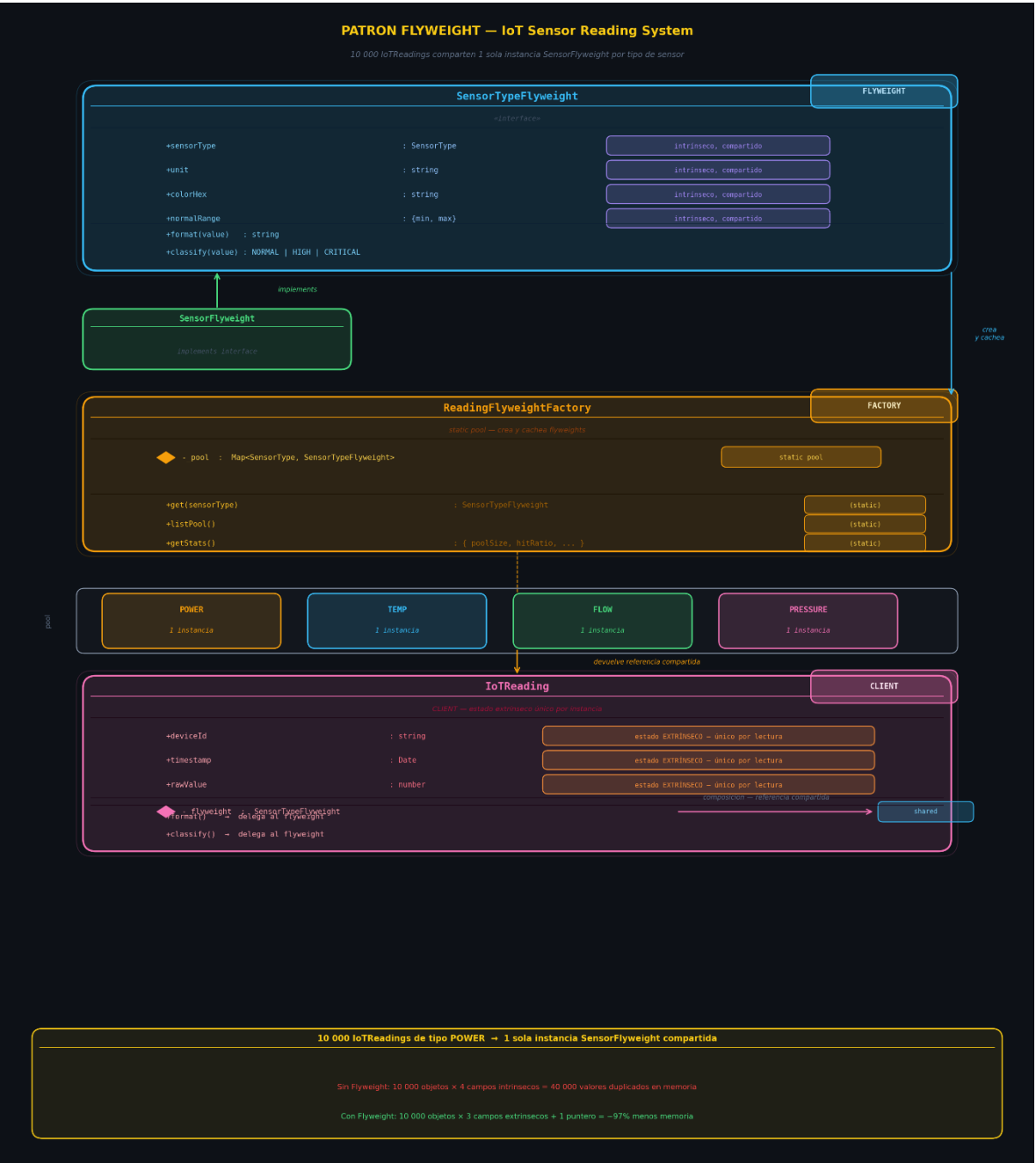
```

class IoTReading {
    readonly deviceId: string; // ← único
    readonly timestamp: Date; // ← único
    readonly rawValue: number; // ← único
    private _flyweight: SensorTypeFlyweight; // ← referencia compartida

    format() → delega al flyweight → "12.50 kW"
    classify() → delega al flyweight → "NORMAL"
}

```


Diagrama Uml



Patrón 7: Flyweight

¿Qué problema resuelve?

El acceso directo a Supabase desde las páginas IoT no tiene control de ningún tipo:

```
// ✗ Sin Proxy – acceso directo, sin seguridad ni caché:  
const { data } = await supabase.from("iot_devices").select("*"); // Llama BD en cada render  
await supabase.from("iot_devices").delete().eq("id", id); // cualquiera puede borrar  
// No hay log, no hay caché, no hay control de roles
```

Con Proxy, el cliente llama a la misma interfaz, pero el proxy intercepta:

```
// ✔ Con Proxy – el cliente no cambia, pero ahora hay control:  
const service = createDeviceService(userId, "VIEWER"); // VIEWER no puede borrar  
const devices = await service.getDevices(userId); // cacheado 30 segundos  
await service.deleteDevice("D1"); // → Error: "ADMIN requerido" (ni llega a Supabase)
```

¿Dónde se usa?

createDeviceService(userId, role) devuelve el proxy configurado Facade puede usarlo: EnergyTradingFacade en lugar de llamar Supabase directo, llama createDeviceService() La página IoT puede reemplazar sus llamadas directas con el proxy

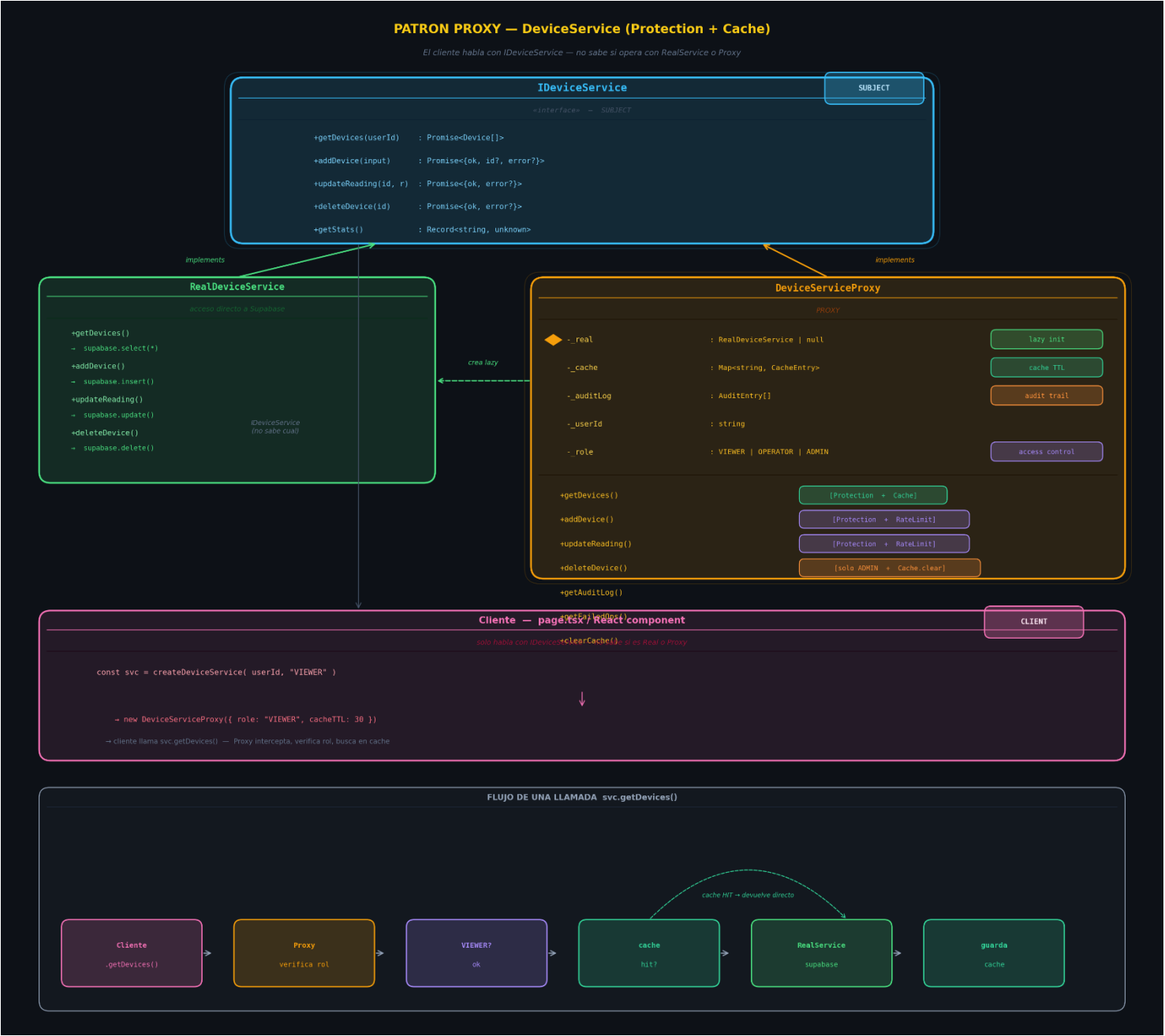
4 tipos de Proxy combinados en una sola clase:

Tipo	Qué hace
Virtual	Crea <code>RealDeviceService</code> solo cuando se llama por primera vez
Protection	Roles <code>VIEWER / OPERATOR / ADMIN</code> — cada operación requiere nivel mínimo
Caching	<code>getDevices()</code> cacheada N segundos; se invalida al agregar/borrar
Logging	Audita cada operación: quién, cuándo, duración, éxito/fallo

Jerarquía de roles:

```
VIEWER → solo getDevices()  
OPERATOR → + addDevice(), updateReading()  
ADMIN → + deleteDevice()
```

Diagrama Uml



PATRON QUE NO USARIA EN MI PROYECTO

Interpreter

¿Por qué no?

El punto clave es que aquí no hay nada que interpretar: no existe un lenguaje, no hay gramática, no hay ambigüedad que resolver. Introducir el patrón Interpreter en este contexto implica construir toda una infraestructura (tokens, árbol de sintaxis, clases por regla) para procesar algo que ya viene estructurado y explícito. Es, en esencia, simular un lenguaje donde no lo hay.

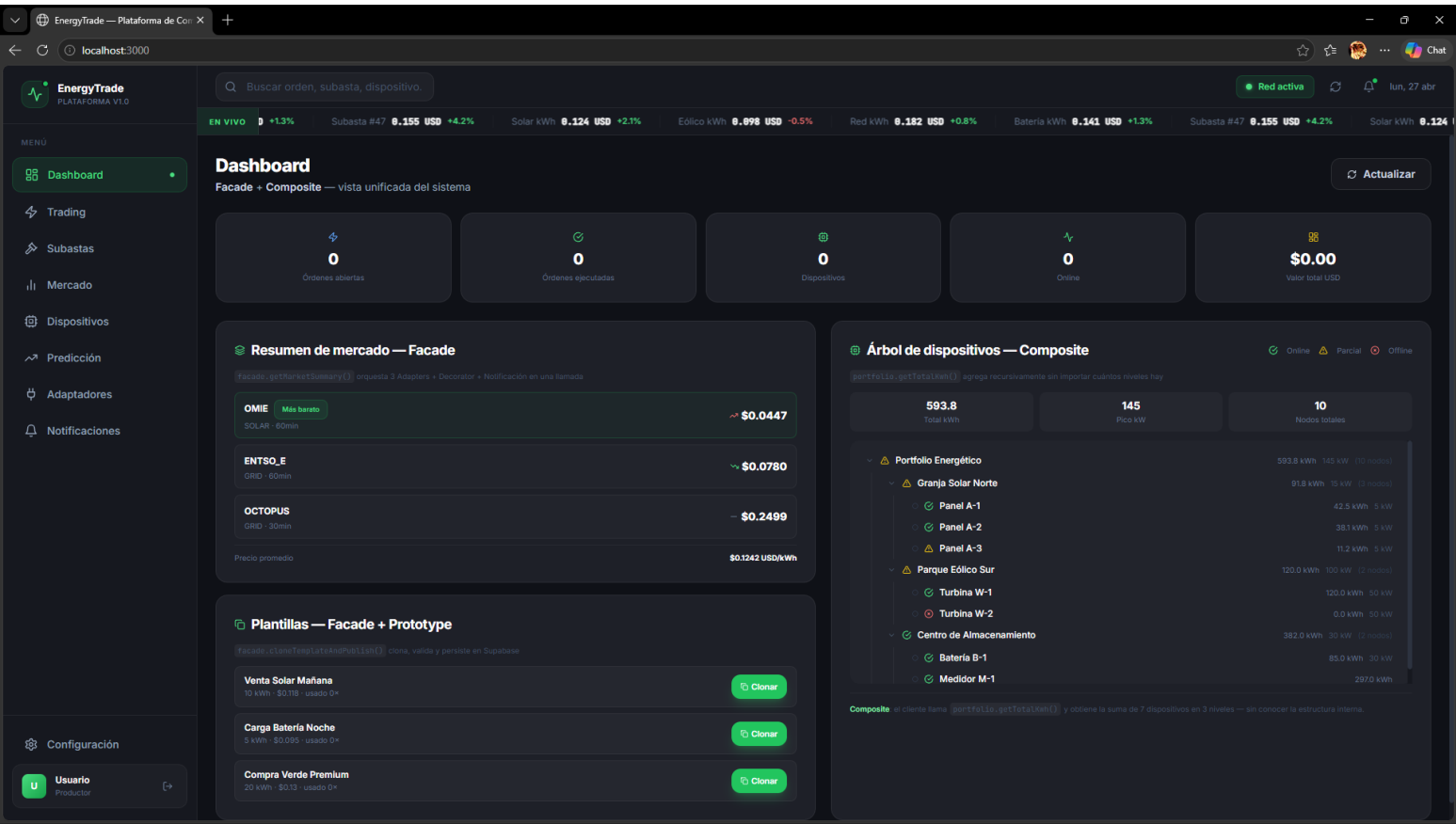
Esto no solo añade complejidad innecesaria, sino que además desplaza el problema hacia un terreno más difícil de mantener. Cada cambio en la “lógica” implica tocar múltiples clases en lugar de modificar una estructura de datos clara o una función directa. Lo que podría resolverse con configuraciones declarativas (JSON) y lógica sencilla termina convertido en un mini compilador interno sin justificación real.

Por eso, más que un patrón inútil, Interpreter es un patrón hiperespecializado que se vuelve contraproducente cuando se aplica fuera de su contexto natural. En proyectos modernos, su uso suele ser una señal de sobreingeniería más que de buen diseño.

FUNCIONAMIENTO DE LA UI DE MI PROYECTO

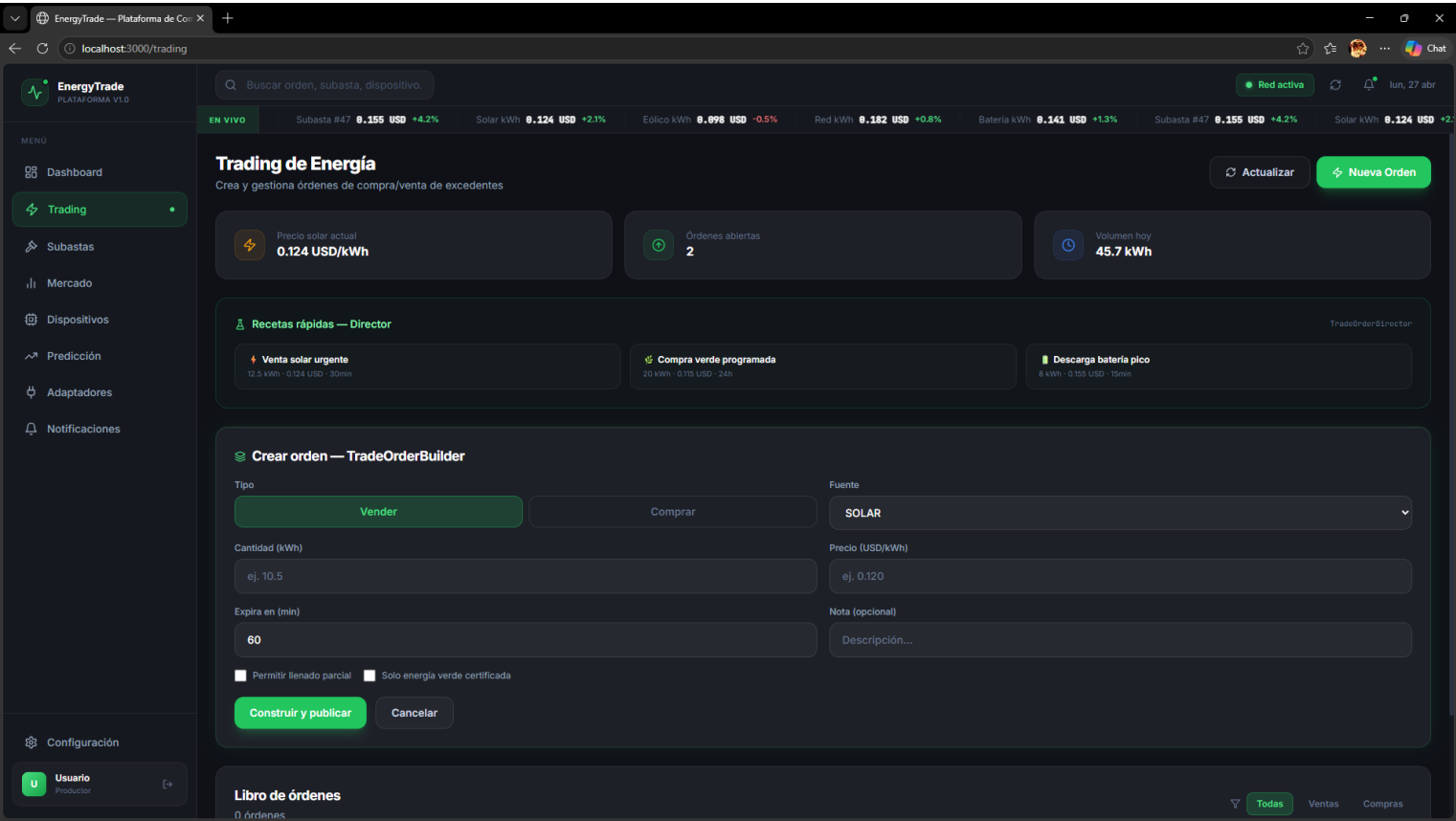
Dashboard

en este se puede ver con facade una vista unificada del sistema del usuario demo



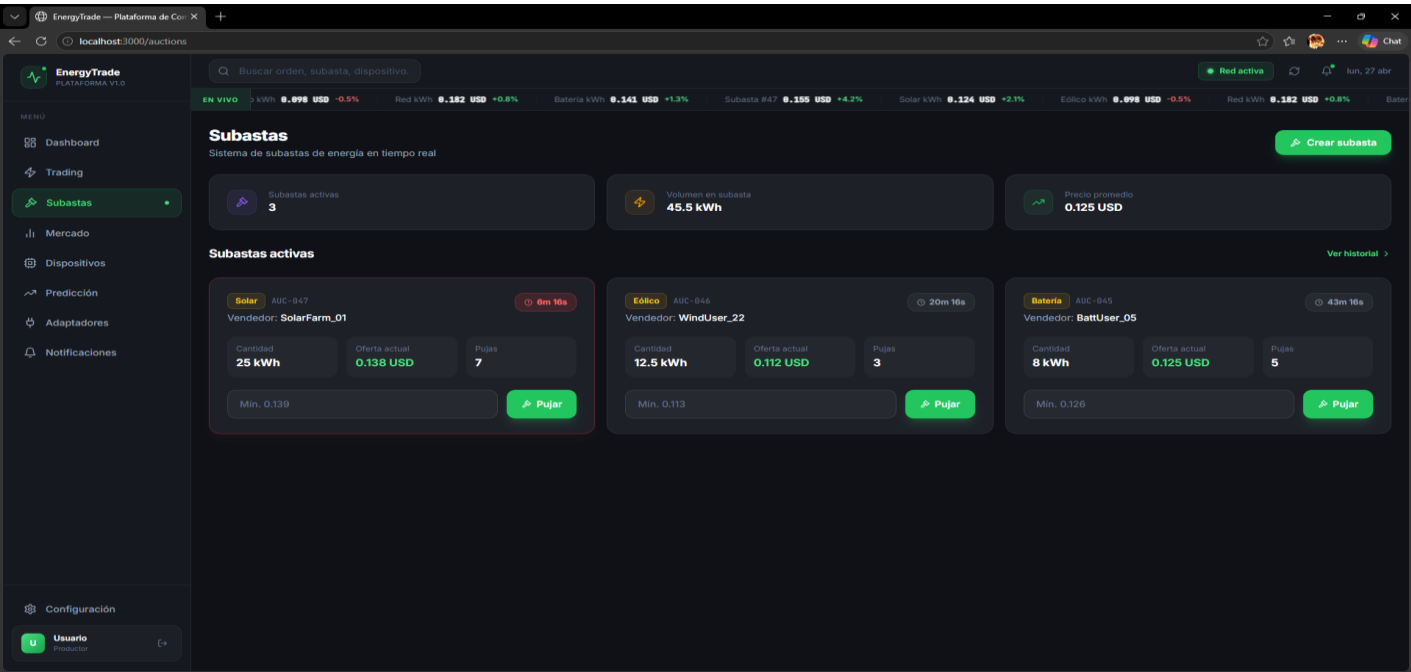
Trading

gestiona las órdenes de compra y venta



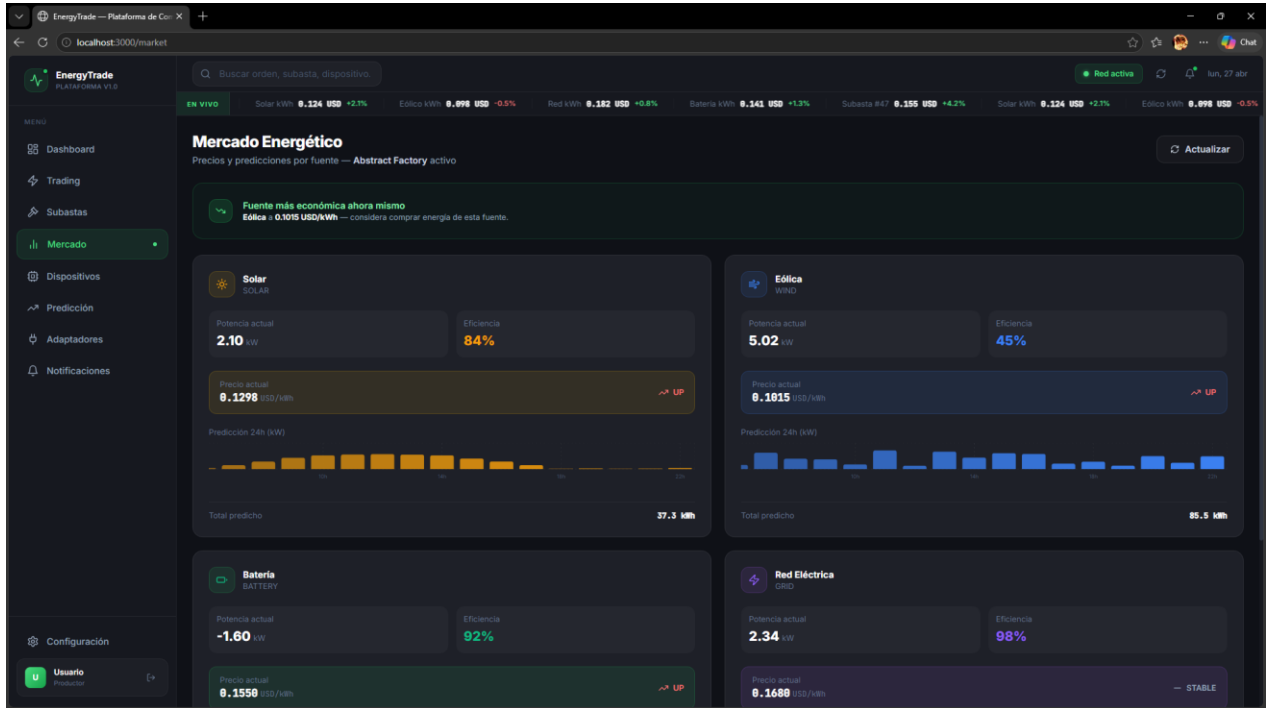
Subastas

Sistema de subasta de energía en tiempo real



Mercado

Muestra el mercado energético de los cuatro tipos de fuentes de energía manejadas



Dispositivos iot

Muestra los dispositivos iot y permite crearlos y añadirlos a una familia

EnergyTrade — Plataforma de Con...

localhost:3000/iot

Dispositivos IoT

Nuevo dispositivo IoT

Nombre *

ej. Panel Solar Terraza

Tipo

Panel Solar

Protocolo

Simulado (desarrollo)

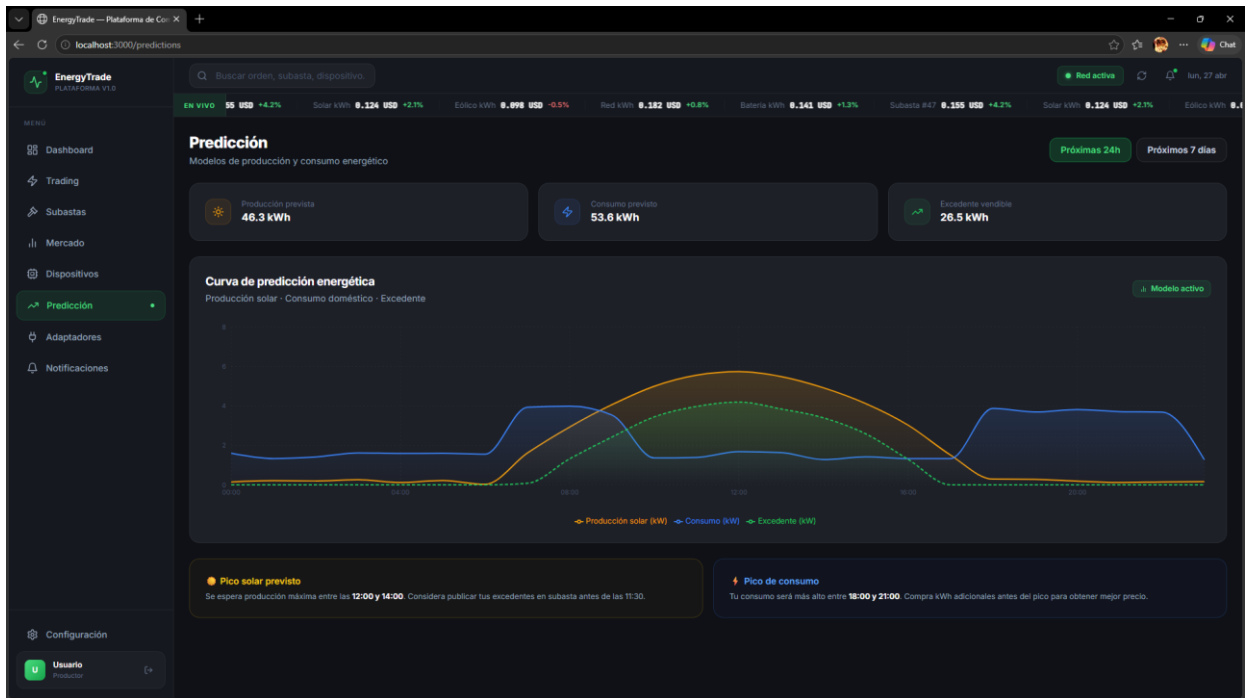
Ubicación (opcional)

ej. Techo Norte

+ Guardar en Supabase Cancelar

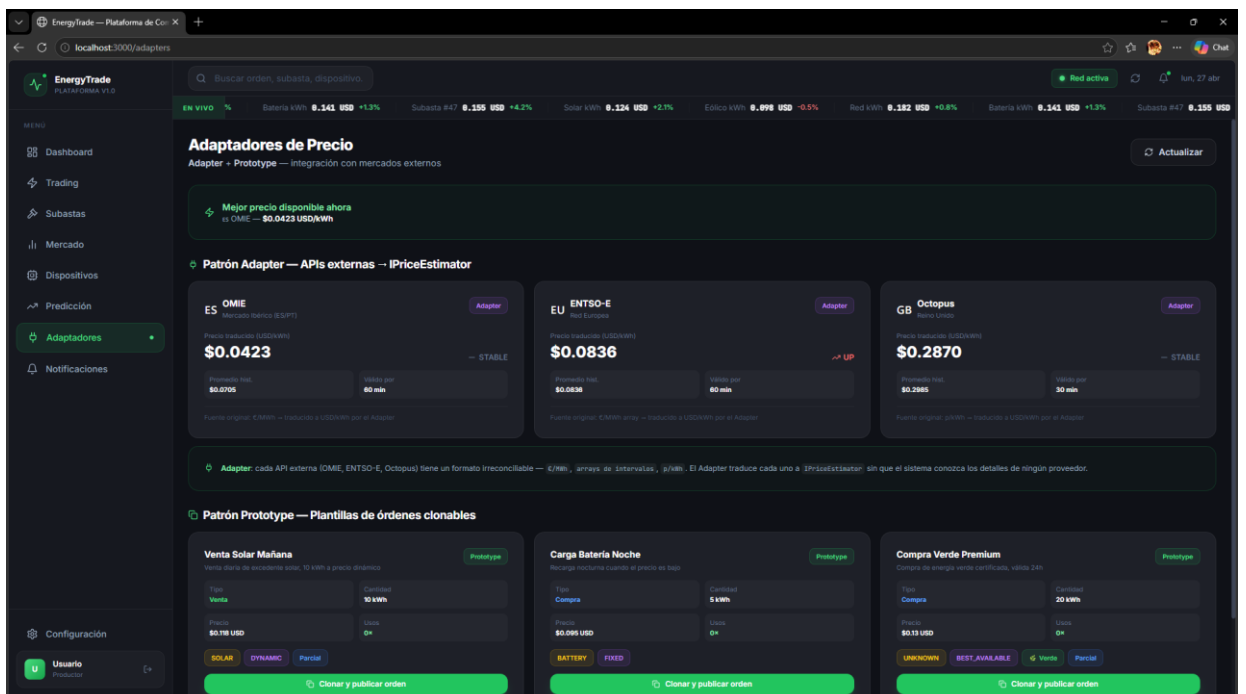
Predicción

Muestra una estimación del consumo versus el consumo



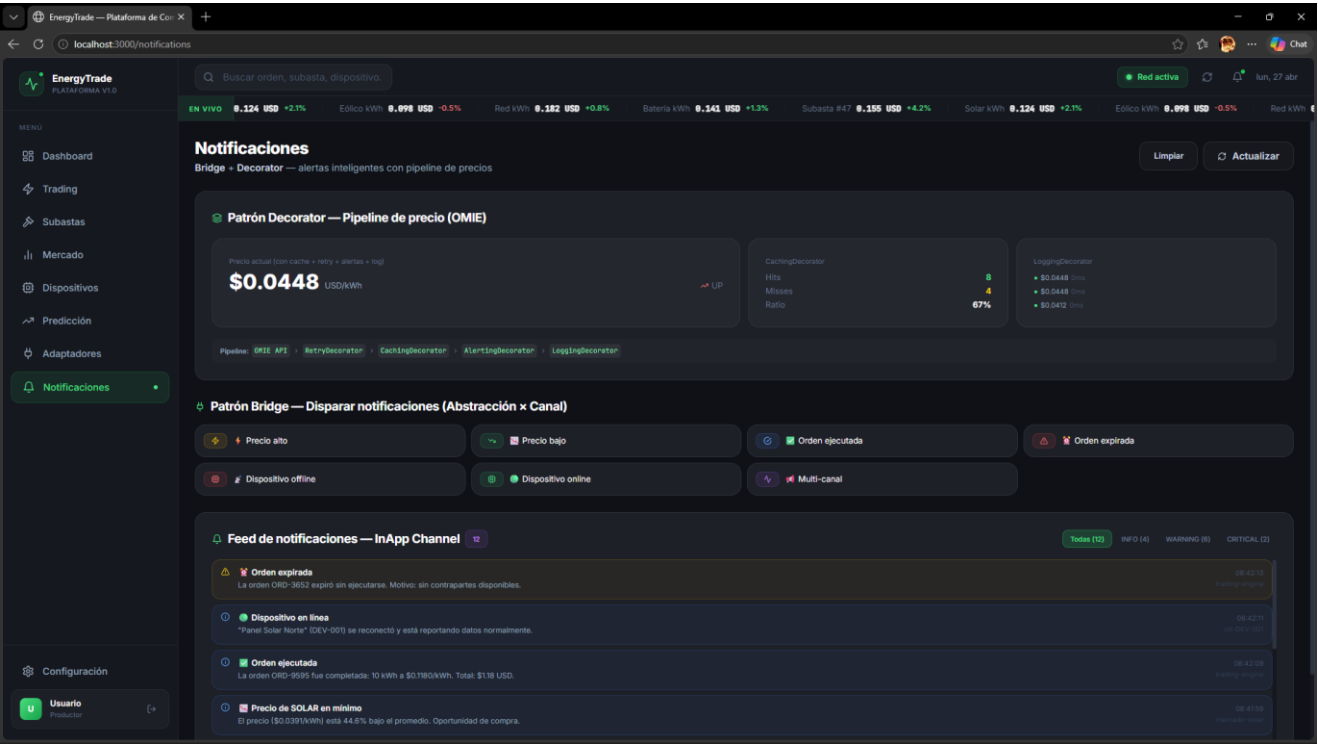
Adaptadores de precio

Adaptadores de los diferentes tipos de precio de energía según la región de la red



Notificaciones

Notifica y alerta todas las novedades que tiene el usuario en diferentes tipos de canales



PATRONES DE COMPORTAMIENTO

Patrón 1: Strategy

¿Qué problema resuelve?

El matching de órdenes de compra/venta de energía necesita algoritmos distintos según el contexto. Sin Strategy, el código sería un bloque gigante con if/else

```
// ✗ Sin Strategy - código frágil, imposible de extender:
function matchOrders(orders, mode) {
  if (mode === "price")      { /* 30 líneas */ }
  else if (mode === "green") { /* 30 líneas */ }
  else if (mode === "fifo")  { /* 30 líneas */ }
  // Para agregar una estrategia nueva → modificar esta función ✗
}
```

Con Strategy, cada algoritmo vive en su propia clase, el Context las intercambia en runtime, y agregar una nueva no toca ningún código existente.

¿Dónde se usa?

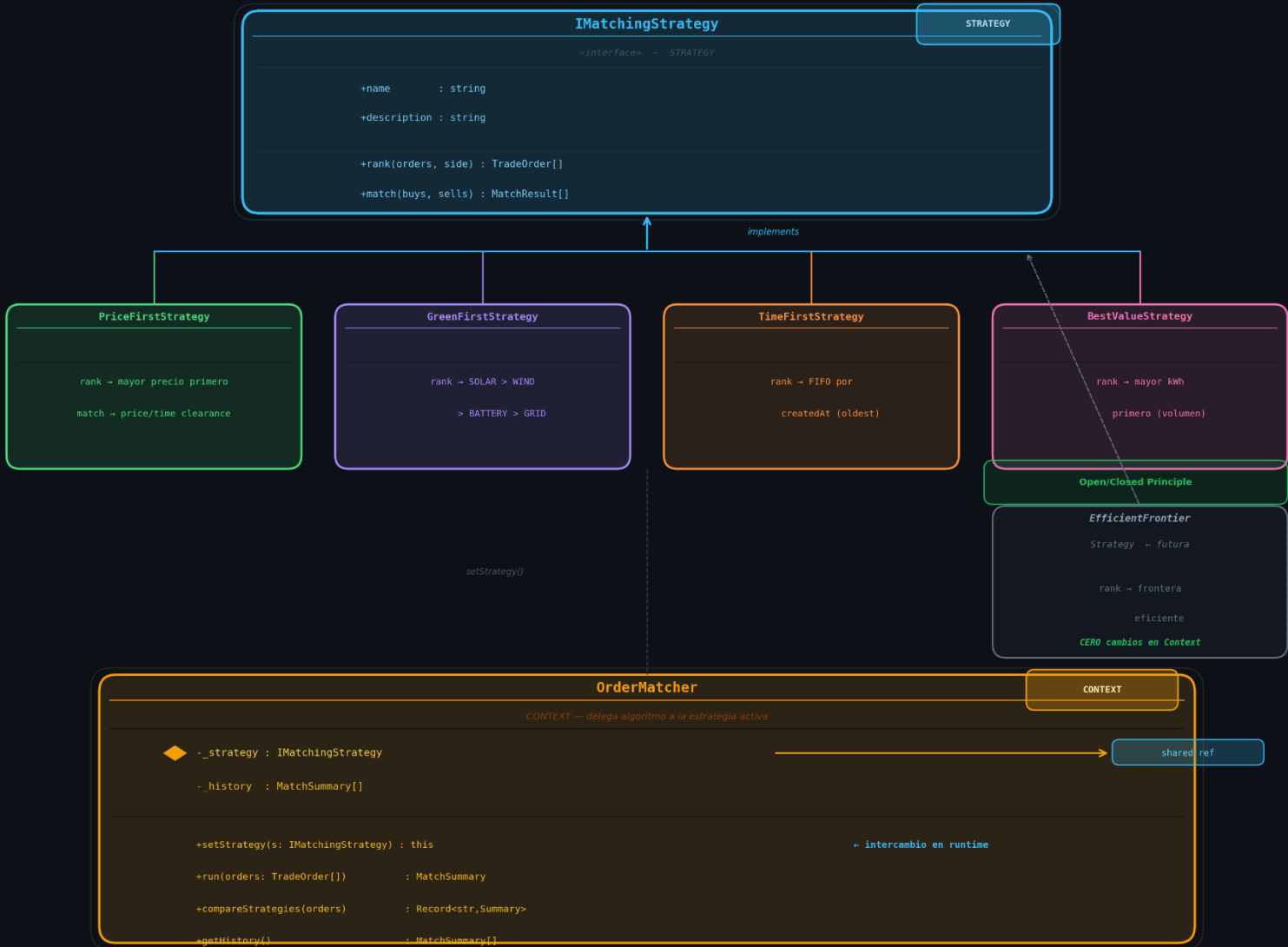
```
interface IMatchingStrategy {
  name: string;
  description: string;
  rank(orders, side): TradeOrder[]; // ordena las órdenes según criterio
  match(buyOrders, sellOrders): MatchResult[]; // empareja compras con ventas
}
```

encapsular cada algoritmo en una clase separada que implementa IMatchingStrategy. El contexto (OrderMatcher) recibe la estrategia en runtime y la delega sin conocer su lógica interna.

Diagrama Uml

PATRON STRATEGY — Order Matching System

El Context (OrderMatcher) delega el algoritmo de matching a la estrategia activa en runtime



USO EN RUNTIME

```
const matcher = new OrderMatcher(new PriceFirstStrategy())

matcher.run(orders) // usa PriceFirst

matcher.setStrategy(new GreenFirstStrategy())

matcher.run(orders) // usa GreenFirst (mismo Context)

matcher.compareStrategies(orders) // prueba todas en paralelo
```

Patrón 2: Command

¿Qué problema resuelve?

Las operaciones sobre dispositivos IoT se llaman directamente

```
// ❌ Sin Command – llamadas directas, sin historial, sin undo:  
await supabase.from("iot_devices").update({ status: "OFFLINE" }).eq("id", "D1");  
// Si fue un error → no hay forma de revertirlo  
// No hay log de qué se hizo ni cuándo  
// No se puede encolar ni programar
```

Con Command, cada operación es un objeto que sabe ejecutarse Y deshacerse

```
// ✅ Con Command – operaciones controladas:  
const cmd = new TurnOffCommand("D1", service);  
await bus.execute(cmd); // ejecuta y guarda en historial  
await bus.undo();       // deshace → el dispositivo vuelve a ONLINE  
await bus.redo();       // rehace  
bus.getHistory();       // ¿quién hizo qué y cuándo?
```

¿Dónde se usa?

```
interface IDeviceCommand {  
  commandType: string;  
  deviceId: string;  
  description: string;  
  execute(): Promise<CommandResult>; // hace la operación  
  undo(): Promise<CommandResult>; // la revierte  
  canUndo(): boolean; // ¿se puede deshacer?  
}
```

encapsula cada operación en un objeto Command que implementa execute() y undo(). El Invoker (DeviceCommandBus) mantiene el historial y controla la ejecución.

```

class DeviceCommandBus {
    private _undoStack: IDeviceCommand[] = [];
    private _redoStack: IDeviceCommand[] = [];
    private _history: CommandRecord[] = [];

    async execute(cmd) {
        const result = await cmd.execute(); // delega al Receiver
        this._history.push({ cmd, result }); // guarda en historial
        if (result.ok) {
            this._undoStack.push(cmd);
            this._redoStack = []; // nuevo cmd invalida redo
        }
    }

    async undo() {
        const cmd = this._undoStack.pop();
        const result = await cmd.undo(); // revierte
        this._redoStack.push(cmd);
        return result;
    }

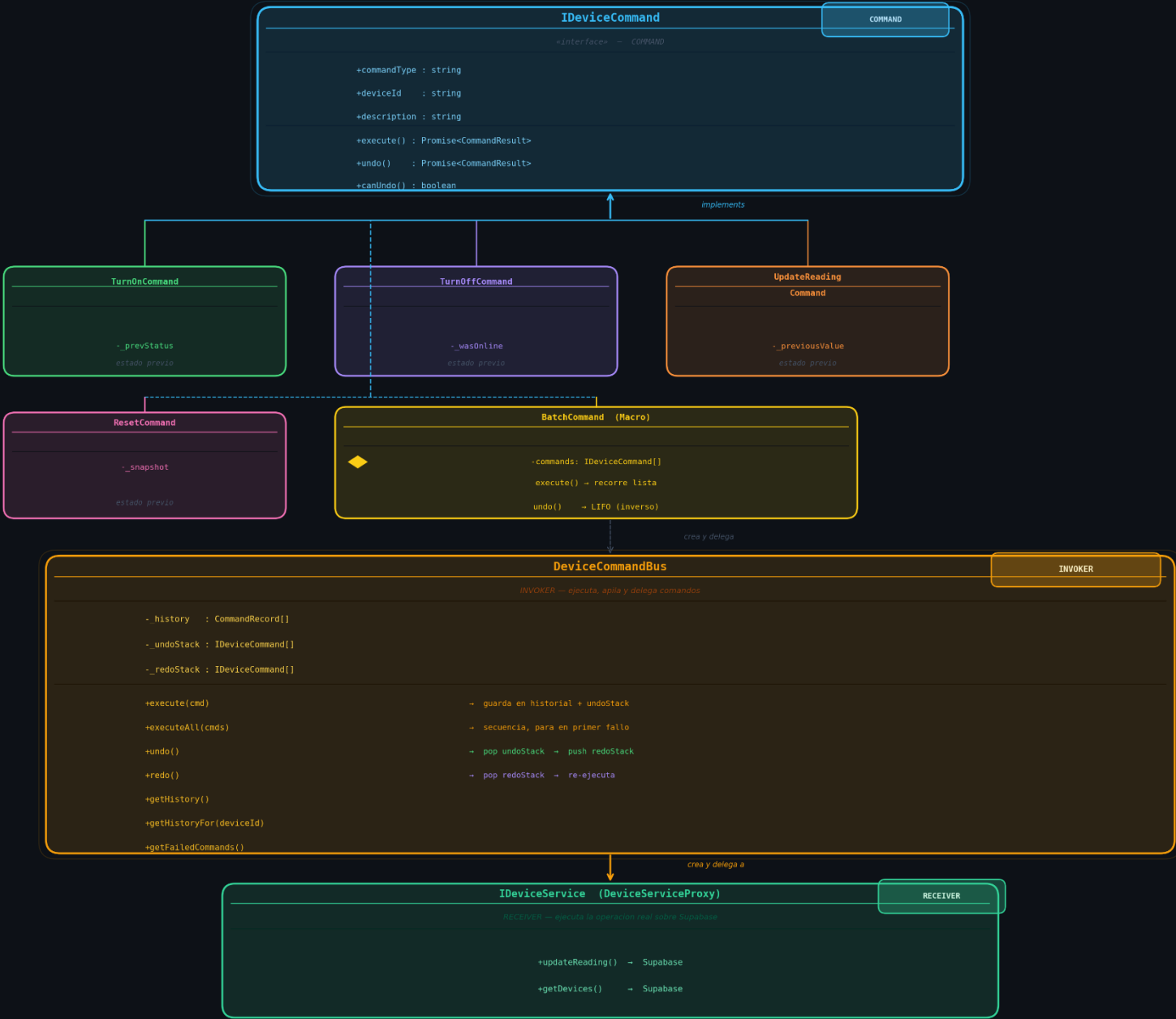
    async redo() {
        const cmd = this._redoStack.pop();
        await this.execute(cmd); // re-ejecuta
    }
}

```

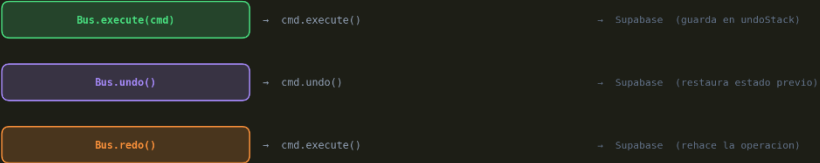
Diagrama Uml

PATRON COMMAND — DeviceCommandBus

Encapsula operaciones como objetos — soporta execute, undo, redo e historial



FLUJO EXECUTE / UNDO / REDO



Patrón 3: Observer

¿Qué problema resuelve?

cuando el precio de mercado cambia, múltiples subsistemas necesitan reaccionar:

- El logger debe registrar el cambio
- El bridge debe enviar una alerta si es un spike
- El matcher debe re-evaluar órdenes pendientes
- La UI debe mostrar una notificación

```
* IMarketObserver: contrato que deben cumplir todos los suscriptores.  
* Cada Observer declara qué tipos de eventos le interesan.  
*/  
export interface IMarketObserver {  
  readonly observerId: string;  
  readonly observerName: string;  
  readonly interestedIn: MarketEventType[]; // filtro declarativo  
  
  onEvent(event: MarketEvent): void | Promise<void>;  
}
```

Sin el patrón Observer, el código encargado de actualizar los datos tendría que llamar directamente a cada uno de estos subsistemas. Esto acoplaría fuertemente los componentes, haciendo que agregar un nuevo suscriptor implique modificar la lógica del emisor.

¿Dónde se usa?

El Subject (MarketEventBus) emite eventos sin conocer a sus suscriptores. Cada Observer decide si le interesa el evento y cómo reaccionar. Completamente desacoplado.

```

export class MarketEventBus {
    private static _instance: MarketEventBus | null = null;

    private readonly _observers    = new Map<string, IMarketObserver>();
    private readonly _eventLog:    MarketEvent[] = [];
    private readonly _maxLogSize:  number;
    private _emitCount             = 0;

    constructor(maxLogSize = 1000) { this._maxLogSize = maxLogSize; }

    /** Singleton para uso global */
    static getInstance(): MarketEventBus {
        if (!MarketEventBus._instance)
            MarketEventBus._instance = new MarketEventBus();
        return MarketEventBus._instance;
    }
}

```

Observadores Concretos:

PriceLogObserver: Almacena y promedia en memoria las cotizaciones recibidas.

AlertObserver: Utiliza el Bridge de notificaciones existente en el proyecto para alertar de desconexiones o picos bruscos de tarifa.

OrderMatchingObserver: Evalúa cuándo re-ejecutar el algoritmo de matching al detectar fluctuaciones de precio que superen un umbral configurado (ej. 2%).

UIFeedObserver: Provee un feed FIFO para consumo de la interfaz de usuario.

```

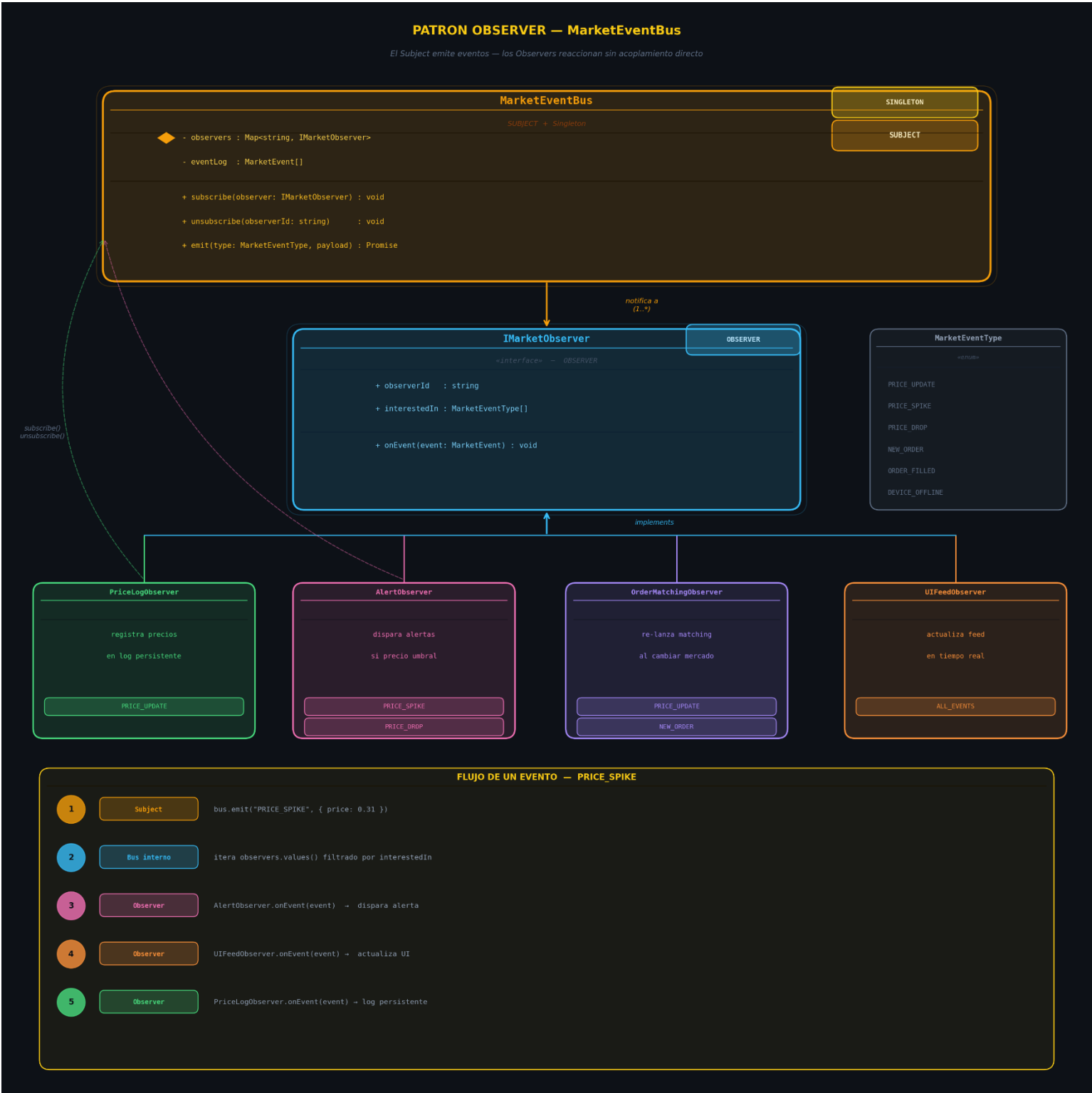
export class UIFeedObserver implements IMarketObserver {
    readonly observerId    = "ui-feed";
    readonly observerName = "Feed de la interfaz";
    readonly interestedIn: MarketEventType[] = [
        "PRICE_UPDATED", "PRICE_SPIKE", "PRICE_DROP",
        "ORDER_MATCHED", "ORDER_EXPIRED", "ORDER_CANCELLED", "ORDER_PUBLISHED",
        "DEVICE_OFFLINE", "DEVICE_ONLINE", "MARKET_SUMMARY_READY",
    ];

    private readonly _feed: MarketEvent[] = [];
    private readonly _maxSize: number;

    constructor(maxSize = 50) { this._maxSize = maxSize; }
}

```


Diagrama Uml



Patrón 4: State

¿Qué problema resuelve?

una orden de trading pasa por varios estados DRAFT → OPEN → MATCHED | EXPIRED | CANCELLED

Sin State: el código usa strings ("OPEN", "MATCHED") y cualquier parte puede asignar cualquier estado sin validación:

```
order.status = "MATCHED"; // ¿pero estaba OPEN? ¿o DRAFT?
```

```
order.status = "CANCELLED"; // ¿puede cancelarse una MATCHED?
```

Con el patrón State, cada estado del ciclo de vida se modela como una clase independiente. Cada clase implementa las operaciones válidas y bloquea con excepciones estructuradas (OrderStateError) cualquier transición ilícita.

¿Dónde se usa?

OrderContext en OrderStateMachine.ts. Envuelve a la orden original, almacena el estado actual y mantiene un historial auditable de transiciones con marcas de tiempo (StateTransition[]).

```
export interface IOrderState {
  readonly statusName: OrderStatus;

  /** DRAFT → OPEN: publica la orden en el mercado */
  publish(ctx: OrderContext): void;

  /** OPEN → MATCHED: empareja la orden con una contraparte */
  match(ctx: OrderContext, meta: MatchMetadata): void;

  /** DRAFT | OPEN → CANCELLED: cancela la orden */
  cancel(ctx: OrderContext, reason: string): void;

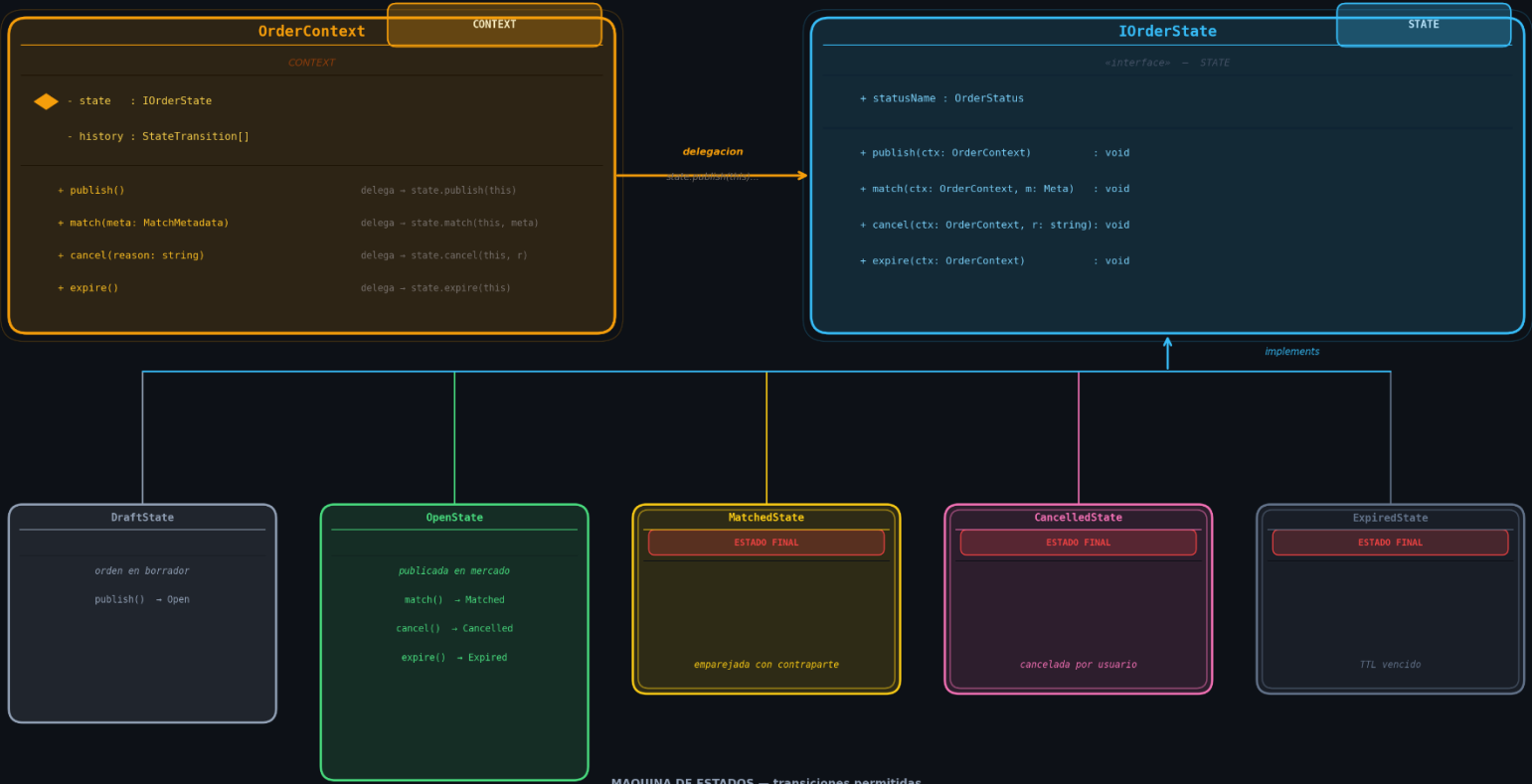
  /** OPEN → EXPIRED: la orden venció sin ejecutarse */
  expire(ctx: OrderContext): void;

  /** Descripción del estado para UI */
  describe(): string;
}
```

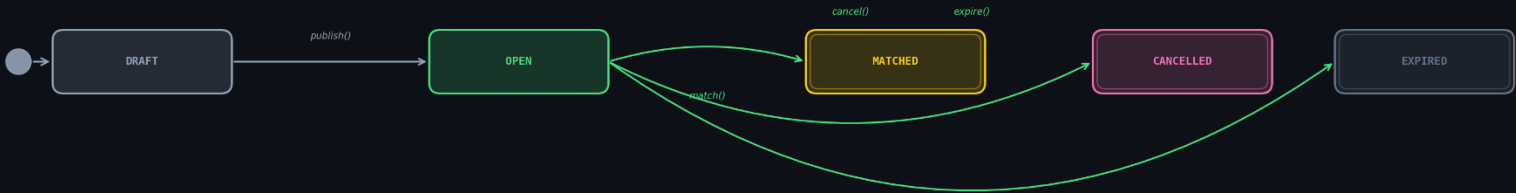
Diagrama Uml

PATRON STATE — Order Lifecycle

OrderContext delega cada accion al estado activo — el estado decide la transicion siguiente



MAQUINA DE ESTADOS — transiciones permitidas



REGLAS DEL PATRON STATE

- Context no conoce los estados concretos — solo habla con IOrderState
- Cada estado decide si la accion es valida y cuál es el estado siguiente
- Estado inválido → lanza error en lugar de hacer nada silenciosamente
- Estados finales (Matched, Cancelled, Expired) ignoran todas las acciones
- `ctx.setState(new OpenState())` ← el estado hijo cambia el contexto

Patrón 5: Template Method

¿Qué problema resuelve?

El sistema necesita generar facturas (comprobantes de transacciones) cuando ocurre un cruce de orden (MatchResult). Aunque el flujo de facturación general es exactamente el mismo para todos los roles del sistema

Sin Template Method, tendríamos lógica dispersa, código duplicado o condicionales complejos (if/else) para modelar las variantes de cobros.

Con Template Method, definimos el esqueleto del algoritmo de facturación en una clase base abstracta (BaseInvoiceProcessor) y delegamos los pasos variables (impuestos, incentivos y comisiones) a subclases concretas.

¿Dónde se usa?

En el archivo BaseInvoiceProcessor.ts, el cual contiene

La Clase Abstracta (AbstractClass):

```
export abstract class BaseInvoiceProcessor {
  protected constructor(public readonly processorType: string) {}

  /**
   * El TEMPLATE METHOD.
   * Define de manera inalterable (final en concepto) el esqueleto
   * del algoritmo para generar la factura.
   */
  public processInvoice(match: MatchResult): Invoice {
    // Step 1: Validar (operación por defecto)
    this.validateMatch(match);

    // Step 2: Calcular importe base (operación por defecto)
    const baseAmount = this.calculateBaseAmount(match);

    // Step 3: Aplicar impuestos (método abstracto, lo decide la subclase)
    const taxAmount = this.applyTaxes(baseAmount);

    // Step 4: Aplicar descuentos/incentivos (hook, opcional de sobrescribir)
    const discountAmount = this.applyDiscountsOrIncentives(match, baseAmount);

    // Step 5: Calcular comisión de la plataforma (método abstracto)
    const platformFee = this.calculatePlatformFee(match, baseAmount);

    // Step 6: Generar número de folio (operación por defecto / hook)
    const invoiceNumber = this.generateInvoiceNumber(match);

    // Step 7: Calcular total final
    const totalDueUsd = parseFloat(
      (baseAmount + taxAmount - discountAmount + platformFee).toFixed(4)
    );

    // Step 8: Redactar observaciones (hook)
    const notes = this.generateNotes(match, discountAmount);

    // Step 9: Retornar factura
    return {
      invoiceNumber,
      buyOrderId: match.buyOrderId,
      sellOrderId: match.sellOrderId,
      energySource: match.energySource,
      matchedKwh: match.matchedKwh,
      pricePerKwh: match.pricePerKwh,
      baseAmount,
      taxAmount,
      discountAmount,
      platformFee,
      totalDueUsd,
      processedAt: new Date(),
      processorType: this.processorType,
      notes,
    };
  }
}
```

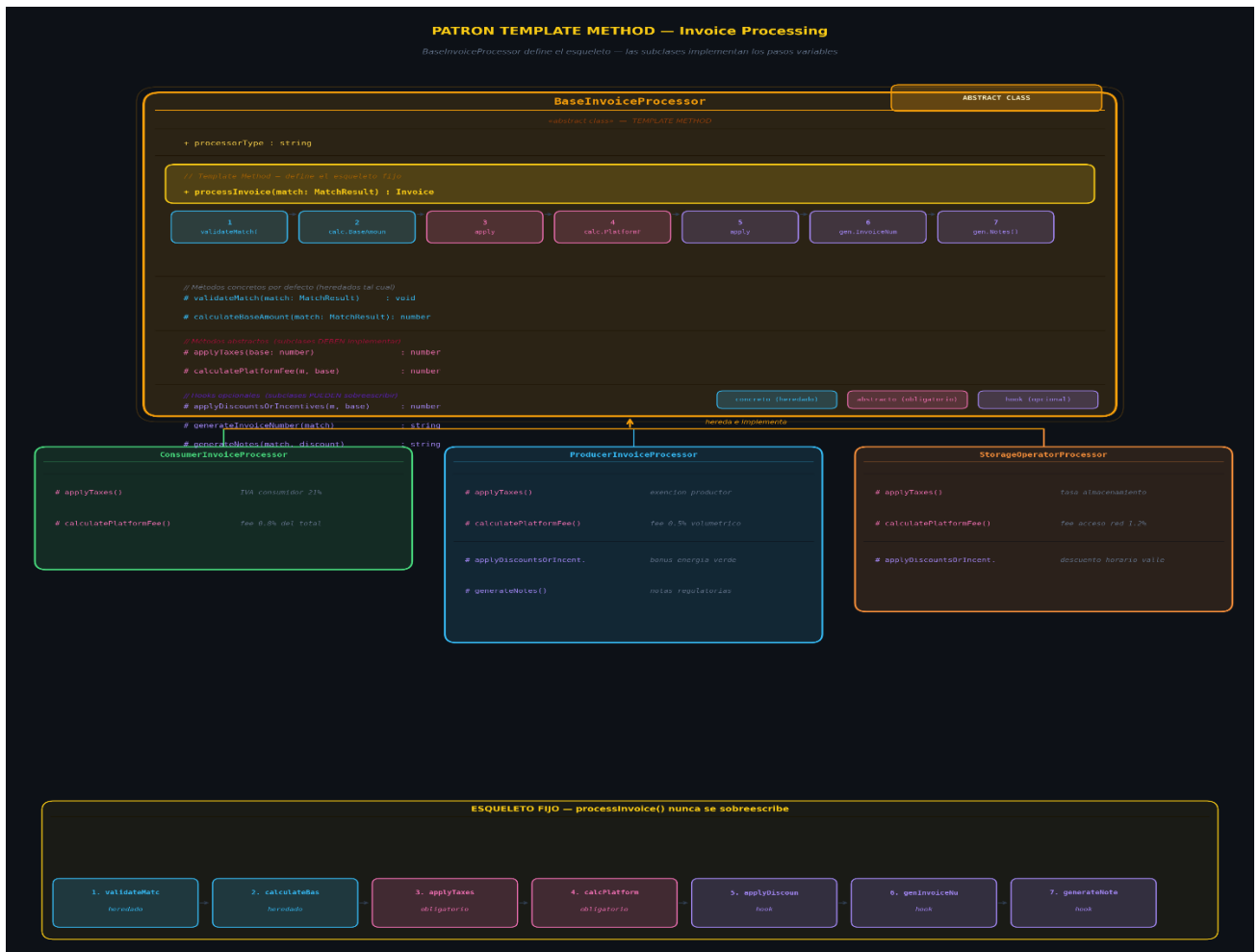
Las Clases Concretas (ConcreteClasses)

```
export class ConsumerInvoiceProcessor extends BaseInvoiceProcessor {
  constructor() {
    super("CONSUMER");
  }

  protected applyTaxes(baseAmount: number): number {
    const networkTax = 0.05; // 5% peajes de red eléctrica
    const VAT = 0.16; // 16% IVA estándar
    return parseFloat((baseAmount * (networkTax + VAT)).toFixed(4));
  }

  protected calculatePlatformFee(_match: MatchResult, baseAmount: number): number {
    const standardFeeRate = 0.015; // 1.5% fee
    return parseFloat((baseAmount * standardFeeRate).toFixed(4));
  }
}
```

Diagrama Uml



Patrón 6: CHAIN OF RESPONSIBILITY

¿Qué problema resuelve?

sensor IoT genera una lectura, hay múltiples reglas de negocio que deben evaluarse antes de emitir una alerta al operador. Sin este patrón, toda la lógica estaría en un único método con if/else anidados que crece indefinidamente con cada nueva regla.

El Decorator ya enriquece y transforma datos (agrega formato, cachea, reintenta). Este patrón cubre únicamente la decisión binaria: ¿debe emitirse esta alerta o no? Son responsabilidades distintas y no se solapan.

¿Dónde se usa?

AlertValidationChain.ts este patrón cubre SOLO la validación/filtrado de alertas (decidir si proceder o detener el flujo). El enriquecimiento y transformación de las alertas ya lo hace el patrón Decorator (PriceDecorator), por lo que no hay solapamiento.

```
* Handler 1 - MaintenanceModeHandler (primer filtro)
* Si el dispositivo está en mantenimiento, ignora toda alerta.
* Regla de negocio: no molestar al operador con alertas programadas.
*/
export class MaintenanceModeHandler extends BaseAlertHandler {
  readonly handlerName = "MaintenanceMode";

  check(request: AlertRequest, log: AlertHandlerResult[]): boolean {
    if (request.alert.inMaintenance) {
      return this.stop(
        log,
        `Dispositivo ${request.alert.deviceId} está en mantenimiento - alerta suprimida.`
      );
    }
    return this.pass(log, "Dispositivo operativo - no está en mantenimiento.");
  }
}
```

```
export class AlertValidationChain {
  private readonly _chain: IAlertHandler;
  private readonly _activeAlerts: Set<string>;

  constructor(initialActiveAlerts: Iterable<string> = []) {
    this._activeAlerts = new Set(initialActiveAlerts);

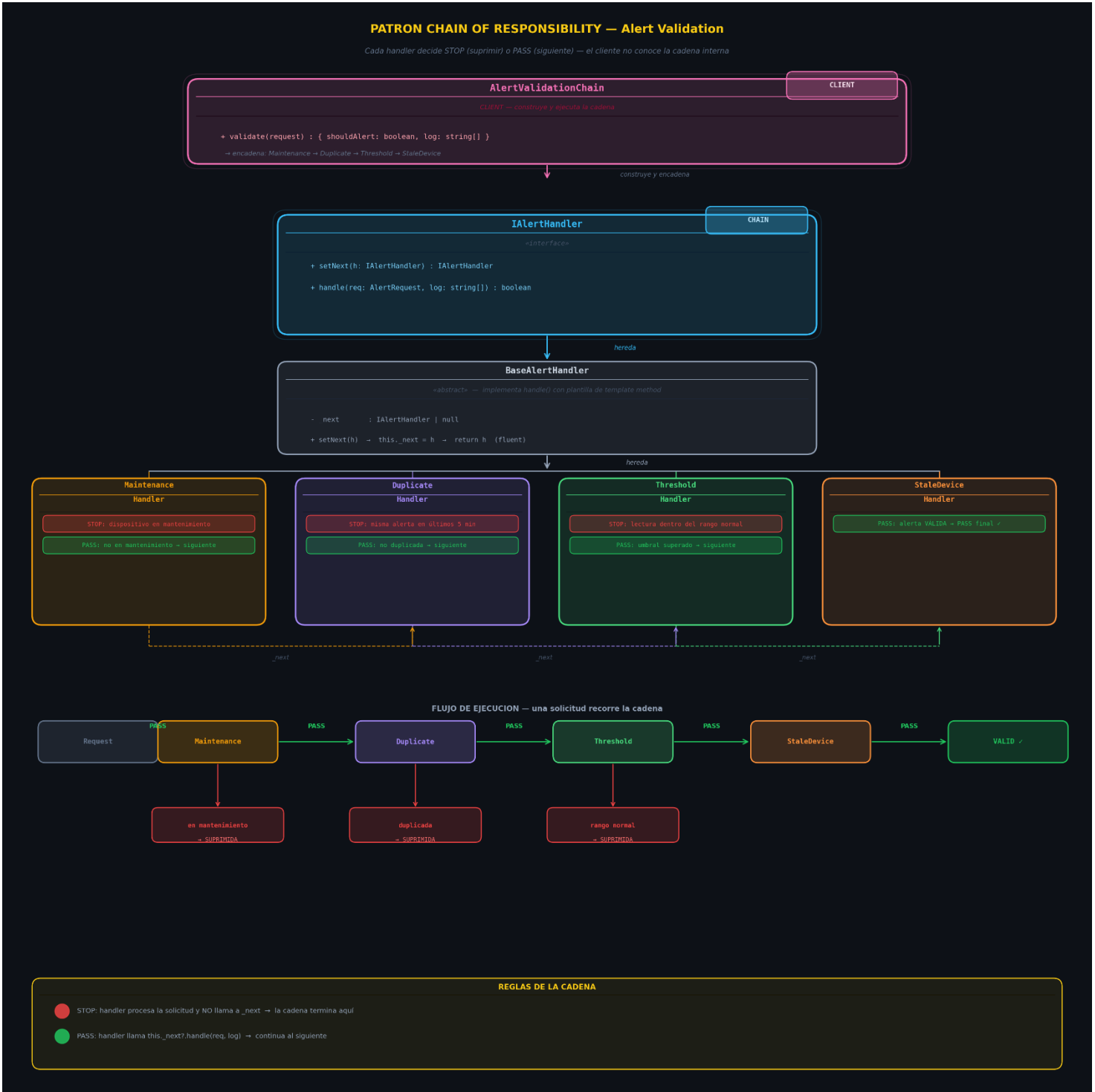
    // Orden: mantenimiento → duplicado → consumo → inactividad
    // (filtros más baratos primero para corto-circuitar antes)
    const maintenance = new MaintenanceModeHandler();
    const duplicate = new DuplicateAlertHandler(this._activeAlerts);
    const threshold = new ThresholdHandler();
    const stale = new StaleDeviceHandler();

    // Encadenamiento fluido
    maintenance.setNext(duplicate).setNext(threshold).setNext(stale);
    this._chain = maintenance;
  }

  /**
   * Ejecuta la cadena y devuelve si la alerta debe emitirse
   * junto con el log completo de decisiones de cada handler.
   */
  validate(request: AlertRequest): { shouldAlert: boolean; log: AlertHandlerResult[] } {
    const log: AlertHandlerResult[] = [];
    const shouldAlert = this._chain.handle(request, log);
    return { shouldAlert, log };
  }

  /** Registra una alerta activa para evitar duplicados */
  markAlertActive(deviceId: string): void {
    this._activeAlerts.add(deviceId);
  }
}
```

Diagrama Uml



Patrón 7: MEDIATOR

¿Qué problema resuelve?

Al procesar un cruce de órdenes de energía, múltiples subsistemas necesitan coordinarse: el motor de matching, el bus de eventos, el procesador de facturas y la cadena de alertas. Sin Mediator, cada componente tendría referencias directas a los demás, creando una red de dependencias de n^2 conexiones.

Con Mediator, cada componente solo conoce al Mediator. Las dependencias quedan reducidas a n conexiones y los Colleagues son reutilizables de forma independiente.

¿Dónde se usa?

TradingMediator.ts coordina 5 Colleagues sin que ninguno se conozca directamente

```
export interface ITradingMediator {  
    /** Procesa un ciclo completo de matching + facturación + eventos */  
    processMatch(request: MediatorMatchRequest): Promise<MediatorMatchResult>;  
  
    /** Valida y procesa una alerta de dispositivo IoT */  
    processAlert(request: MediatorAlertRequest): Promise<MediatorAlertResult>;  
  
    /** Notifica un evento de orden directamente al bus */  
    notifyOrderEvent(type: "ORDER_PUBLISHED" | "ORDER_CANCELLED", payload: OrderPayload): Promise<void>;  
}
```



```

* TradingMediator: coordina los subsistemas del mercado de energía.
*
* Colleagues gestionados:
*   - OrderMatcher:      ejecuta el matching según la estrategia activa
*   - MarketEventBus:    emite eventos del ciclo de vida de las órdenes
*   - DeviceCommandBus:  encola comandos automáticos de dispositivos IoT
*   - AlertValidationChain: decide si una alerta de dispositivo procede
*   - BaseInvoiceProcessor: genera facturas según el rol del participante
*
* El Mediator:
*   1. Recibe la solicitud de matching del cliente
*   2. Delega al OrderMatcher para obtener los MatchResult
*   3. Para cada match, delega al InvoiceProcessor según el rol
*   4. Emite los eventos ORDER_MATCHED al MarketEventBus
*   5. Devuelve el resumen completo al cliente
*/
export class TradingMediator implements ITradingMediator {

    private readonly _matcher:      OrderMatcher;
    private readonly _eventBus:     MarketEventBus;
    private readonly _commandBus:   DeviceCommandBus;
    private readonly _alertChain:   AlertValidationChain;
    private readonly _processors: Record<ParticipantRole, BaseInvoiceProcessor>;

    private _processedCount = 0;

    constructor(options: {
        matcher?: OrderMatcher;
        eventBus?: MarketEventBus;
        commandBus?: DeviceCommandBus;
        alertChain?: AlertValidationChain;
    } = {}) {
        // Inyección de dependencias con defaults razonables (DI friendly)
        this._matcher = options.matcher ?? new OrderMatcher();
        this._eventBus = options.eventBus ?? MarketEventBus.getInstance();
        this._commandBus = options.commandBus ?? new DeviceCommandBus();
        this._alertChain = options.alertChain ?? new AlertValidationChain();

        // Cada rol del mercado tiene su propio procesador de facturas
        this._processors = {
            CONSUMER:      new ConsumerInvoiceProcessor(),
            PRODUCER:      new ProducerInvoiceProcessor(),
            STORAGE_OPERATOR: new StorageOperatorInvoiceProcessor(),
        };
    }
}

```

Diagrama Uml



Patrón 8: VISITOR

¿Qué problema resuelve?

El árbol Composite (EnergyDevice / EnergyGroup) es estable y no debe modificarse. Sin embargo, el sistema necesita ejecutar operaciones adicionales sobre sus nodos (auditoría, fiscalidad, eficiencia) sin contaminar esas clases. Agregar métodos a IEnergyNode cada vez que surge una nueva operación violaría el Principio Abierto/Cerrado (OCP).

como las clases existentes no se pueden tocar, se usa un acceptVisitor() externo que aplica discriminación de tipos (instanceof) en un único lugar aislado, en lugar de dispersar ese instanceof por todo el código cliente.

¿Dónde se usa?

EnergyNodeVisitor.ts Tres visitantes concretos operan sobre el árbol sin tocar las clases originales:

```
/**
 * IEnergyVisitor: define una operación de visita por cada tipo
 * concreto de nodo que existe en el árbol Composite.
 * Agregar un nuevo Visitor = nueva clase, sin tocar los nodos.
 */
export interface IEnergyVisitor {
    readonly visitorName: string;

    visitDevice(device: EnergyDevice): void;
    visitGroup(group: EnergyGroup): void;
}
```

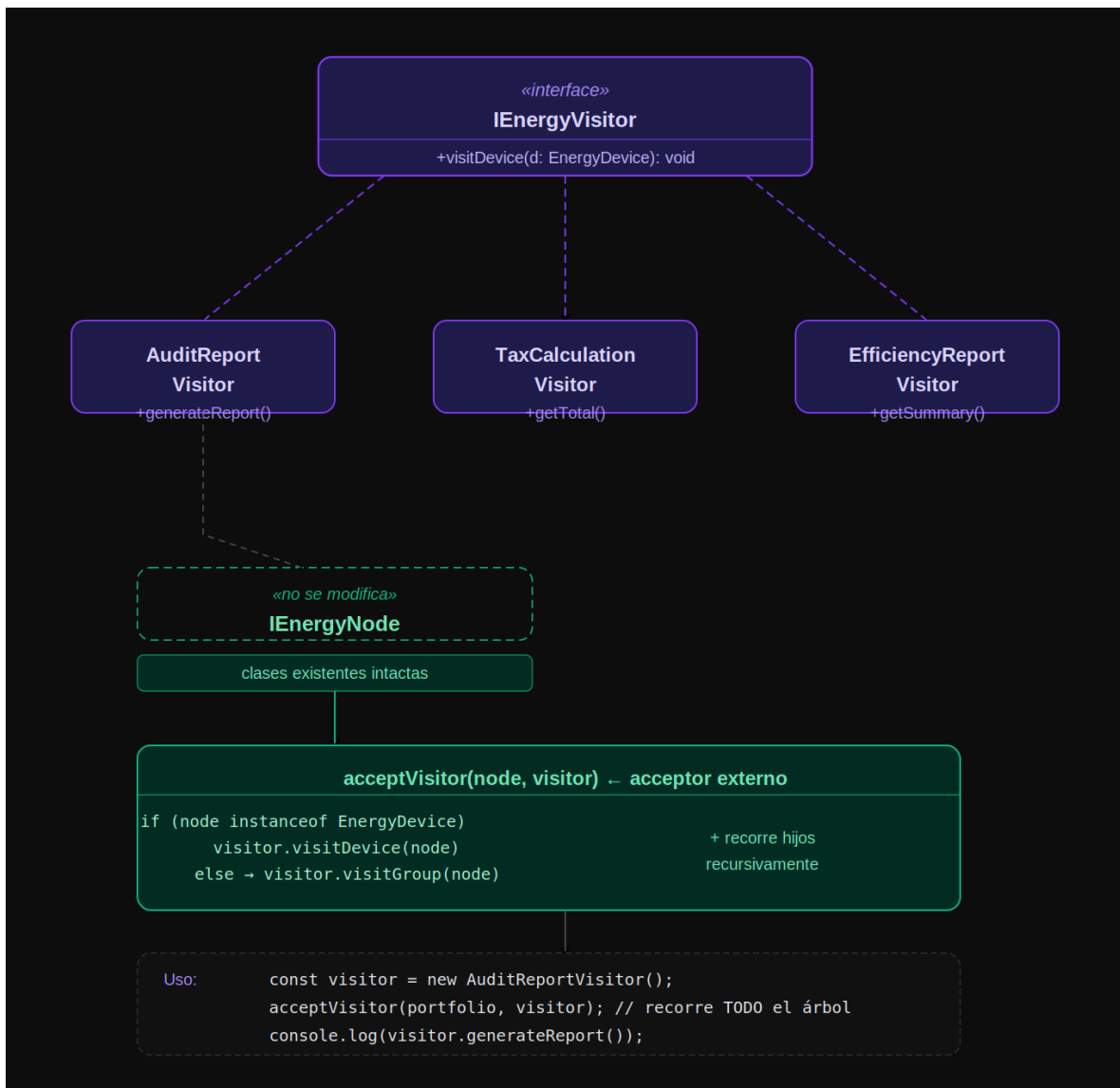
```
* AuditReportVisitor: genera un reporte de cumplimiento normativo.
* Un dispositivo es "compliant" si está ONLINE y genera > 0 kWh.
* Un grupo es "compliant" si todos sus hijos están en ONLINE.
*/
export class AuditReportVisitor implements IEnergyVisitor {
    readonly visitorName = "AuditReport";
    private readonly _entries: AuditEntry[] = [];

    visitDevice(device: EnergyDevice): void {
        const online = device.getStatus() === "ONLINE";
        const producing = device.getTotalKwh() > 0;
        const compliant = online && producing;

        this._entries.push({
            nodeType: "DEVICE",
            name: device.getName(),
            status: device.getStatus(),
            totalKwh: device.getTotalKwh(),
            peakKw: device.getPeakPowerKw(),
            compliant,
            issue: !online
                ? "Dispositivo fuera de línea"
                : !producing
                    ? "Sin generación registrada"
                    : undefined,
        });
    }

    visitGroup(group: EnergyGroup): void {
        const compliant = group.getStatus() === "ONLINE";
        this._entries.push({
            nodeType: "GROUP",
            name: group.getName(),
            status: group.getStatus(),
            totalKwh: group.getTotalKwh(),
            peakKw: group.getPeakPowerKw(),
            compliant,
            issue: !compliant
                ? `Grupo en estado ${group.getStatus()} - revisar hijos`
                : undefined,
        });
    }
}
```

Diagrama Uml



Patrón 9: MEMENTO

¿Qué problema resuelve?

El libro de órdenes activas (OrderBook) es mutable: el operador puede agregar órdenes, limpiarlas en lote o cambiar el precio de referencia. Sin Memento, cualquier operación destructiva es irreversible. El patrón permite hacer snapshots del estado completo y volver atrás cuando se comete un error.

máximo 5 snapshots retenidos (FIFO). El Caretaker solo accede a metadatos del Memento, nunca al estado interno.

¿Dónde se usa?

OrderBookSnapshot.ts Memento guarda/restaura el estado completo del libro de órdenes y el precio de mercado. No cubre undo/redo infinito: el Caretaker mantiene un máximo de 5 snapshots por limitación intencional del demo

```
export class OrderBookMemento {
  // Estado interno - solo accesible desde OrderBook (convención de módulo)
  /** @internal */ readonly _orders:    readonly TradeOrder[];
  /** @internal */ readonly _marketPrice: number;
  /** @internal */ readonly _isLocked:   boolean;

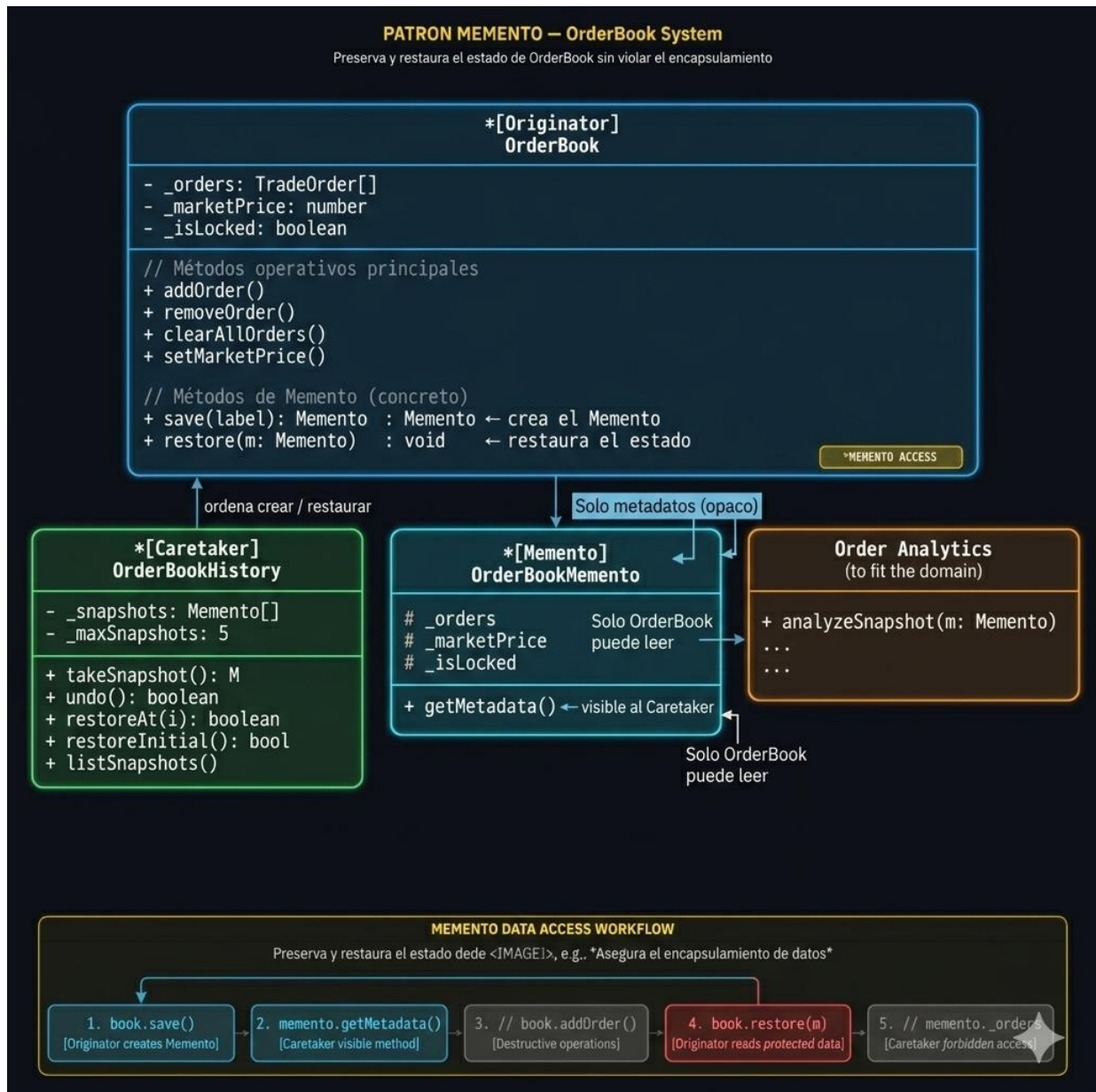
  readonly createdAt: Date;
  readonly label:     string;

  /** @internal Solo el Originator debe construir Mementos */
  constructor(
    orders:    readonly TradeOrder[],
    marketPrice: number,
    isLocked:   boolean,
    label:      string,
  ) {
    // Copia profunda de las órdenes para evitar mutación compartida
    this._orders = Object.freeze([...orders]);
    this._marketPrice = marketPrice;
    this._isLocked = isLocked;
    this.createdAt = new Date();
    this.label = label;
  }
}
```

```
/**
 * Crea un snapshot del estado actual.
 * El label ayuda al Caretaker a identificar cuándo se tomó.
 */
save(label = "snapshot"): OrderBookMemento {
  return new OrderBookMemento(
    this._orders,
    this._marketPrice,
    this._isLocked,
    label,
  );
}

/**
 * Restaura el estado desde un Memento.
 * Solo el Originator puede leer el estado interno del Memento.
 */
restore(memento: OrderBookMemento): void {
  this._orders = [...memento._orders];
  this._marketPrice = memento._marketPrice;
  this._isLocked = memento._isLocked;
}
```

Diagrama Uml



Patrón 10: Iterator

Por qué NO implemente el patrón Iterator en este proyecto

TypeScript/JavaScript implementa el protocolo Iterable de forma nativa a nivel del runtime. Toda colección del proyecto ya es iterable sin código adicional

```
// Arrays – ya son Iterable<T>
for (const order of this._orders) { ... }           // OrderBook
for (const child of this._children) { ... }         // EnergyGroup
for (const [id, obs] of this._observers) { ... }    // MarketEventBus

// Desestructuración y spread – operan sobre iteradores nativos
const sorted = [...this._orders].sort(...);
const ids    = this._orders.map(o => o.id);
const open   = this._orders.filter(o => o.status === "OPEN");
```

Implementar un ConcreteIterator personalizado replicaría exactamente lo que el motor JS ya hace internamente, sin ningún beneficio adicional.

El GoF Iterator resuelve un problema que aquí no existe

El patrón fue diseñado en los años 90 para lenguajes (C++, Java de la época) donde:

- Las colecciones internas no tenían un mecanismo de recorrido estándar.
- Se necesitaba abstraer estructuras como listas enlazadas, árboles o grafos detrás de una interfaz uniforme.

En este proyecto todas las colecciones internas son arrays planos (TradeOrder[], IEnergyNode[], MarketEvent[]). El problema de "ocultar la estructura interna" ya está resuelto por la propia interfaz de los módulos.

El árbol Composite ya se recorre mejor con Visitor

El único lugar donde un Iterator podría tener sentido es el árbol EnergyGroup → EnergyDevice. Sin embargo:

El patrón Visitor (ya implementado en EnergyNodeVisitor.ts) recorre ese mismo árbol y además aplica operaciones tipadas por nodo.

Un Iterator sobre el árbol solo daría un cursor de recorrido, pero sin poder distinguir si el nodo actual es un Leaf o un Composite, que es exactamente la distinción que Visitor ya maneja.

Tener ambos sería redundante.

Aspecto	Valor real aportado
Nuevo comportamiento	✗ Ninguno — <code>for...of</code> , <code>.map()</code> , <code>.filter()</code> ya cubren todo
Abstracción útil	✗ Las colecciones ya son opacas tras sus APIs públicas
Reducción de código cliente	✗ El código cliente quedaría más verbose , no menos
Líneas de código nuevo	~80-120 líneas de <code>IIterator</code> , <code>ConcreteIterator</code> , <code>IterableAggregate</code> que no aportan lógica de dominio

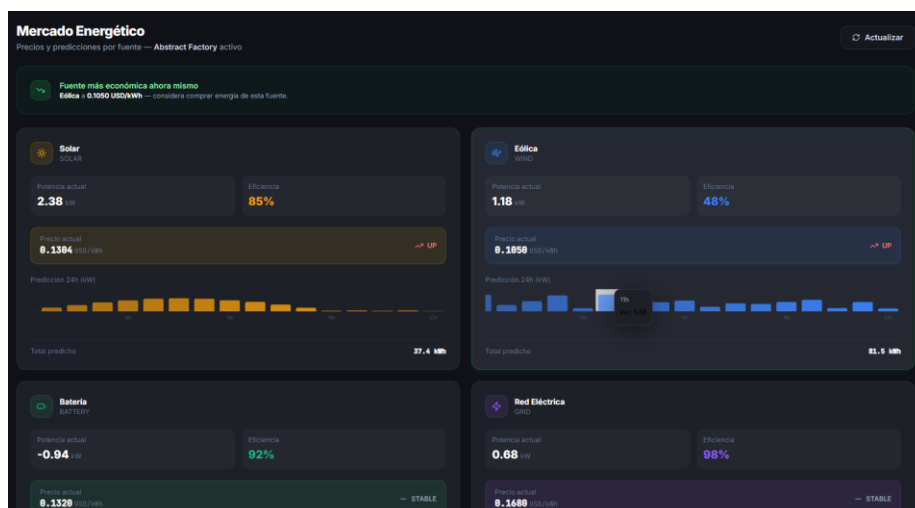
Implementarlo solo para completar el catálogo GoF sería complejidad ceremonial: código que existe para existir, no para resolver un problema. En una codebase de producción, eso es deuda técnica desde el primer commit.

5 resultados del proyecto

1. Dashboard unificado con vista en tiempo real

Se logró una vista centralizada del sistema que muestra órdenes activas, dispositivos IoT, resumen de mercado y plantillas disponibles en una sola pantalla, sin que el componente React conozca la complejidad interna de los subsistemas.

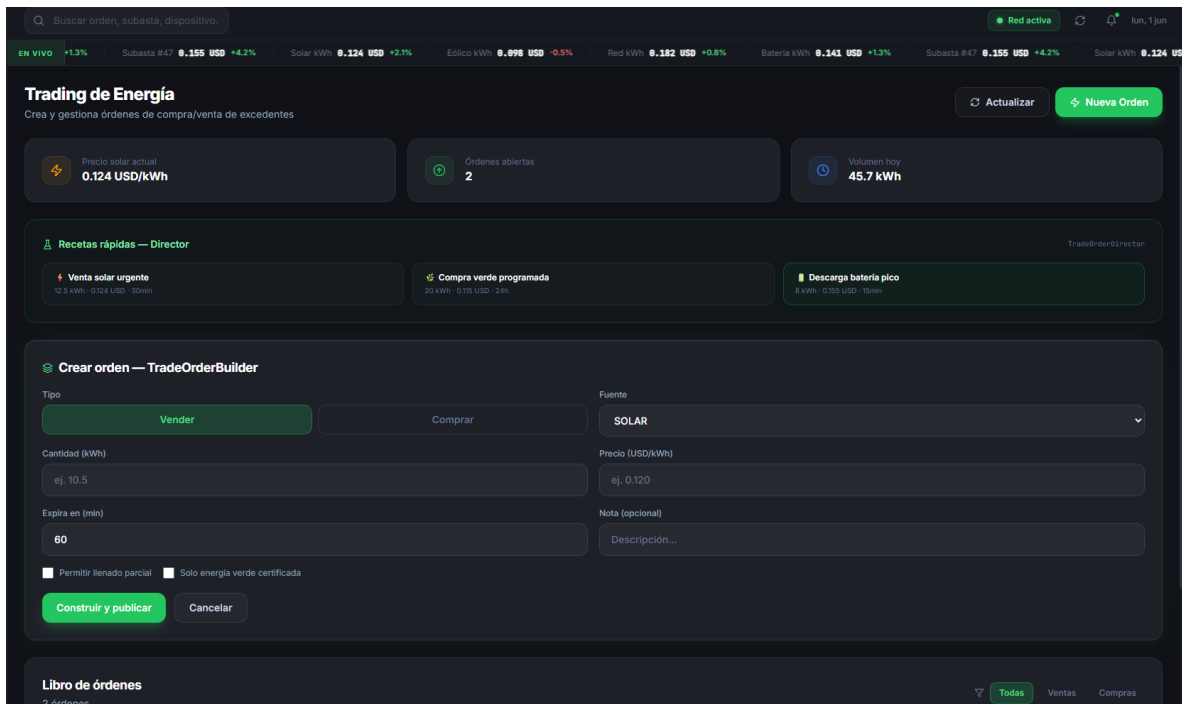
Facade, Composite, Singleton



2. Sistema de creación y publicación de órdenes de trading

Se implementó un flujo completo de creación de órdenes con API fluida (Builder), recetas rápidas predefinidas (Director) y clonación de plantillas existentes (Prototype), permitiendo crear y publicar órdenes en un solo paso desde la UI.

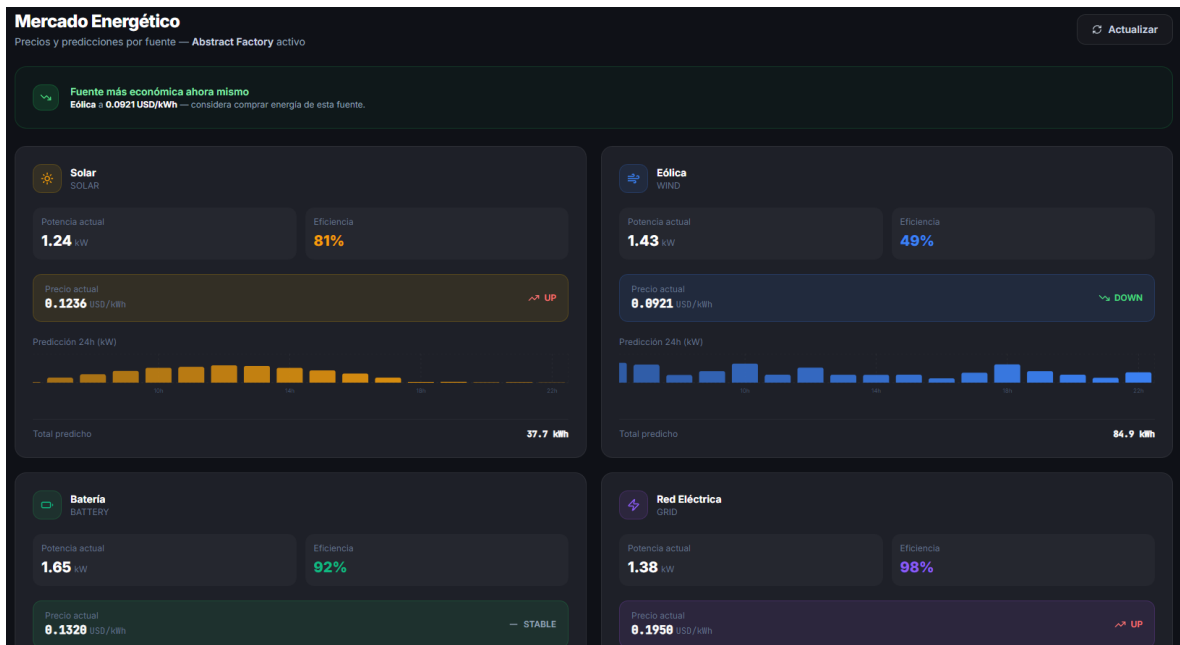
Builder, Prototype, Facade



3. Mercado energético con precios de múltiples proveedores externos

Se integraron tres proveedores de precios incompatibles (OMIE en €/MWh, ENTSO-E y Octopus) bajo una interfaz estándar única, con caché TTL, logging, alertas de spike y reintentos automáticos apilados como capas independientes.

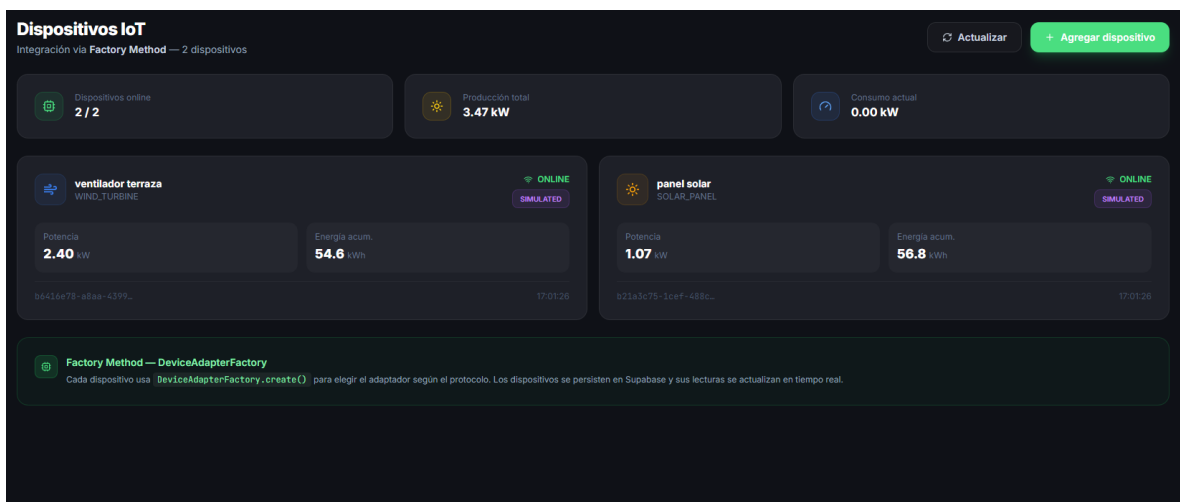
Adapter, Decorator, Singleton



4. Gestión de dispositivos IoT con control de acceso por roles

Se implementó la creación de dispositivos IoT usando Factory Method (protocolos MQTT, REST, Simulated), Abstract Factory (familias Solar, Eólica, Batería, Red) y Proxy con control de roles (VIEWER / OPERATOR / ADMIN), caché de 30 s y auditoría de cada operación.

Factory Method, Abstract Factory, Proxy, Flyweight



The image shows a dark-themed modal window titled "Nuevo dispositivo IoT" with a close button (X) in the top right corner. The form contains the following fields:

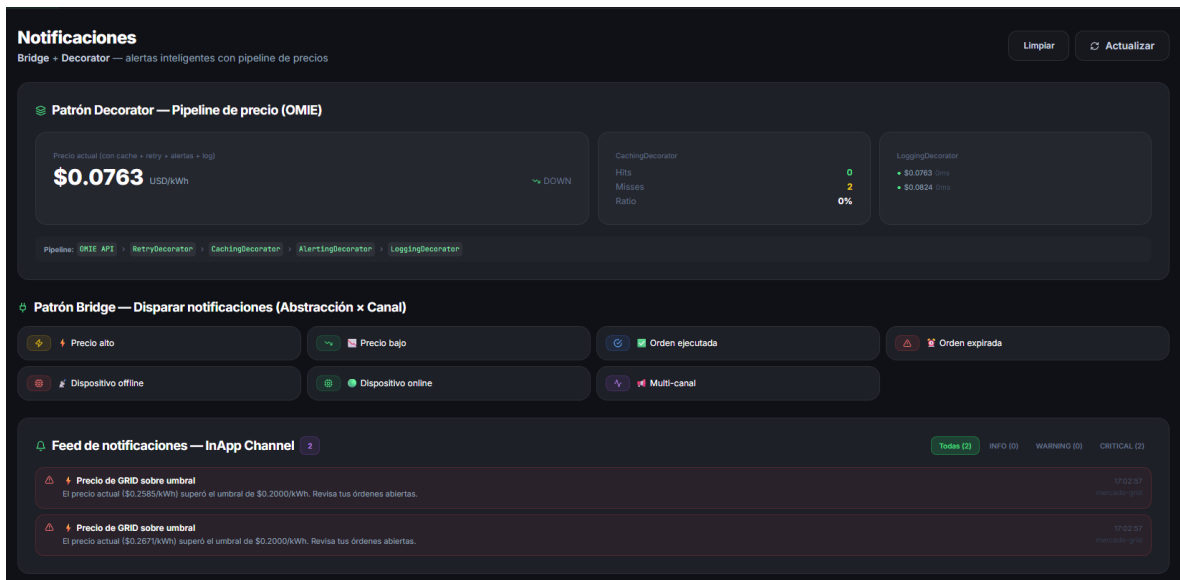
- Nombre ***: A text input field with the placeholder text "ej. Panel Solar Terraza".
- Tipo**: A dropdown menu with a sun icon and the selected option "Panel Solar".
- Protocolo**: A dropdown menu with a green arrow icon and the selected option "Simulado (desarrollo)".
- Ubicación (opcional)**: A text input field with the placeholder text "ej. Techo Norte".

At the bottom of the form are two buttons: a green button labeled "+ Guardar en Supabase" and a gray button labeled "Cancelar".

5. Sistema de notificaciones multi-canal con alertas inteligentes

Se entregó un sistema que dispara alertas de precio, estado de órdenes y dispositivos offline a través de cuatro canales (consola, in-app, email, webhook), reduciendo de 12 clases posibles a 7 mediante Bridge, y usando el Decorator como disparador automático de alertas al superar umbrales de precio.

Bridge, Decorator



10 Evidencias por consola de los últimos patrones

1 — Observer: suscripción y disparo de evento PRICE_SPIKE

Ruta: /notifications

Pasos:

1. Abre la página de Notificaciones.
2. En el panel "Patrón Decorator — Pipeline de precio (OMIE)", haz clic en **Actualizar**.
3. Espera a que el precio supere el umbral configurado (spikeThreshold: 0.18).

```

[MarketEventBus] Initializing singleton instance...
[MarketEventBus] Instance created. Observers registered: 0

[MarketEventBus] emit('PRICE_UPDATED') → source: SOLAR price: $0.1242/kWh
[MarketEventBus] Notifying 4 observers filtered by interestedIn...
[PriceLogObserver] onEvent ← PRICE_UPDATED | stored. avg: $0.1198 (12 samples)
[UIFeedObserver] onEvent ← PRICE_UPDATED | feed updated (50 entries max)
[OrderMatchingObserver] onEvent ← PRICE_UPDATED | Δprice: 1.2% – below threshold (2%), skip re-match
[AlertObserver] onEvent ← PRICE_UPDATED | not interested, skipped

[MarketEventBus] emit('PRICE_SPIKE') → source: SOLAR price: $0.2340/kWh threshold: $0.18
[MarketEventBus] Notifying 4 observers filtered by interestedIn...
[PriceLogObserver] onEvent ← PRICE_SPIKE | stored. avg: $0.1401 (13 samples)
[UIFeedObserver] onEvent ← PRICE_SPIKE | feed updated → 1 new entry
[AlertObserver] onEvent ← PRICE_SPIKE | threshold exceeded! Δ+30.0%
→ PriceAlertNotifier.alertPriceSpike('SOLAR', 0.2340, 0.18)
→ InAppChannel.deliver({ title: 'Precio alto', level: 'CRITICAL', source: 'SOLAR', price: 0.2340 })
[OrderMatchingObserver] onEvent ← PRICE_SPIKE | Δprice: 30.0% – re-matching pending orders...
→ OrderMatcher.run(orders) triggered by price change

[MarketEventBus] emitCount: 2 | totalObserverCalls: 7

```

Evidencia 2 — Observer: múltiples observers reaccionando al mismo evento

Ruta: /notifications

Pasos:

1. En el panel "Patrón Bridge — Disparar notificaciones", se activa manualmente el botón **"Precio alto"** o **"Multi-canal"**.
2. Observa el feed InApp.

```

[MarketEventBus] emit('DEVICE_OFFLINE') → deviceId: 'D1-Panel-A' name: 'Panel Solar A'
[MarketEventBus] Iterating 4 registered observers...
[PriceLogObserver] interestedIn: [PRICE_UPDATED] – not interested, skipped
[UIFeedObserver] interestedIn: [ALL_EVENTS] – onEvent() called
→ feed.unshift({ id: 'evt-091', type: 'DEVICE_OFFLINE', title: 'Panel Solar A desconectado', ts: '14:32:07' })
→ feed.length: 23 entries
[AlertObserver] interestedIn: [PRICE_SPIKE, DEVICE_OFFLINE] – onEvent() called
→ DeviceNotifier.notifyDeviceOffline('D1-Panel-A', 'Panel Solar A')
→ InAppChannel.deliver({ title: 'Dispositivo offline', level: 'WARNING', deviceId: 'D1-Panel-A' })
[OrderMatchingObserver] interestedIn: [PRICE_UPDATED, NEW_ORDER] – not interested, skipped

[MarketEventBus] Event DEVICE_OFFLINE dispatched to 2/4 observers (2 filtered)

```

Evidencia 3 — Strategy: cambio de algoritmo de matching en runtime

Ruta: /auctions (Subastas)

Pasos:

1. Se abre la página de Subastas con al menos una subasta activa visible.
2. Se Entra a /trading y crea una orden de compra con fuente SOLAR a precio de mercado.
3. Se Vuelve a Subastas y haz clic en "**nueva orden**" sobre la subasta solar activa.

```
[OrderMatcher] Active strategy: PriceFirstStrategy
[OrderMatcher] run() called → 3 buy orders, 4 sell orders

[PriceFirstStrategy] rank(buyOrders, 'BUY')
→ sorted by pricePerkWh DESC (highest willing to pay first)
[0] ORD-B-441 | BUY | 10.0 kWh | $0.1380/kWh | SOLAR
[1] ORD-B-438 | BUY | 5.5 kWh | $0.1250/kWh | WIND
[2] ORD-B-435 | BUY | 8.0 kWh | $0.1100/kWh | SOLAR
[PriceFirstStrategy] rank(sellOrders, 'SELL')
→ sorted by pricePerkWh ASC (cheapest seller first)
[0] ORD-S-201 | SELL | 12.0 kWh | $0.1180/kWh | SOLAR
[1] ORD-S-198 | SELL | 6.0 kWh | $0.1240/kWh | SOLAR
[2] ORD-S-195 | SELL | 9.0 kWh | $0.1300/kWh | WIND
[3] ORD-S-190 | SELL | 4.0 kWh | $0.1410/kWh | BATTERY

[PriceFirstStrategy] match()
MATCH #1 → ORD-B-441 × ORD-S-201 | 10.0 kWh @ $0.1180/kWh | total: $1.1800
MATCH #2 → ORD-B-438 × ORD-S-198 | 5.5 kWh @ $0.1240/kWh | total: $0.6820
ORD-B-435 → no compatible seller at $0.1100 – unmatched

[OrderMatcher] Summary → matched: 2 | unmatched: 1 | totalVolume: 15.5 kWh | totalValue: $1.862
```

Evidencia 4 — Strategy: comparación entre estrategias

Ruta: /trading

Pasos:

1. Se crea dos órdenes de venta con diferentes fuentes (una SOLAR, una WIND) al mismo precio.
2. Se crea una orden de compra que pueda cruzar con ambas.
3. Ejecuta el matching usando primero **PriceFirst** y luego **GreenFirst** desde el libro de órdenes.

```

[OrderMatcher] setStrategy(GreenFirstStrategy) — strategy swapped at runtime
[OrderMatcher] run() called → same 3 buy orders, same 4 sell orders

```

```

[GreenFirstStrategy] rank(sellOrders, 'SELL')
→ sorted by energySource priority: SOLAR(1) > WIND(2) > BATTERY(3) > GRID(4)
[0] ORD-S-201 | SELL | 12.0 kWh | $0.1180/kWh | SOLAR ← priority 1
[1] ORD-S-198 | SELL | 6.0 kWh | $0.1240/kWh | SOLAR ← priority 1
[2] ORD-S-195 | SELL | 9.0 kWh | $0.1300/kWh | WIND ← priority 2
[3] ORD-S-190 | SELL | 4.0 kWh | $0.1410/kWh | BATTERY ← priority 3
[GreenFirstStrategy] rank(buyOrders, 'BUY')
→ sorted by energySource priority (green buyers first), then kWh DESC
[0] ORD-B-441 | BUY | 10.0 kWh | $0.1380 | SOLAR
[1] ORD-B-435 | BUY | 8.0 kWh | $0.1100 | SOLAR
[2] ORD-B-438 | BUY | 5.5 kWh | $0.1250 | WIND

```

```

[GreenFirstStrategy] match()
MATCH #1 → ORD-B-441 × ORD-S-201 | 10.0 kWh @ $0.1180/kWh | SOLAR×SOLAR
MATCH #2 → ORD-B-435 × ORD-S-198 | 6.0 kWh @ $0.1240/kWh | SOLAR×SOLAR (partial fill)
ORD-B-438 → no WIND seller at $0.1250 — unmatched

```

```

[OrderMatcher] compareStrategies() result logged to history[1]
[OrderMatcher] Strategy=GreenFirst vs PriceFirst → same volume (15.5 kWh), different pairing order

```

Evidencia 5 — Command: execute, undo y redo sobre un dispositivo IoT

Ruta: /iot (Dispositivos IoT)

Pasos:

1. Se Selecciona cualquier dispositivo online de la lista.
2. Se Ejecuta la acción **"Apagar"** (TurnOffCommand) — el estado cambia a OFFLINE.
3. Se Ejecuta **"Deshacer"** — el dispositivo vuelve a ONLINE.
4. Se Ejecuta **"Rehacer"** — el dispositivo vuelve a OFFLINE.

```

[DeviceCommandBus] Initialized | undoStack: 0 | redoStack: 0

[DeviceCommandBus] execute(TurnOffCommand) → deviceId: 'D1-Panel-A'
[TurnOffCommand] execute() called
→ _wasOnline = true (saved for undo)
→ DeviceServiceProxy.updateReading('D1-Panel-A', { status: 'OFFLINE' })
→ Supabase UPDATE iot_devices SET status='OFFLINE' WHERE id='D1-Panel-A' ✓
[TurnOffCommand] result: { ok: true, deviceId: 'D1-Panel-A', newStatus: 'OFFLINE' }
[DeviceCommandBus] pushed to undoStack | undoStack: 1 | redoStack: 0 (cleared)

[DeviceCommandBus] undo()
[TurnOffCommand] undo() called
→ _wasOnline: true → restoring to ONLINE
→ DeviceServiceProxy.updateReading('D1-Panel-A', { status: 'ONLINE' })
→ Supabase UPDATE iot_devices SET status='ONLINE' WHERE id='D1-Panel-A' ✓
[TurnOffCommand] undo result: { ok: true, restoredStatus: 'ONLINE' }
[DeviceCommandBus] moved to redoStack | undoStack: 0 | redoStack: 1

[DeviceCommandBus] redo()
[TurnOffCommand] execute() called (redo)
→ Supabase UPDATE iot_devices SET status='OFFLINE' WHERE id='D1-Panel-A' ✓
[DeviceCommandBus] undoStack: 1 | redoStack: 0

```

Evidencia 6 — Command: BatchCommand sobre múltiples dispositivos

Ruta: /iot

Pasos:

1. Se Selecciona 2 o más dispositivos a la vez.
2. Se Ejecuta una acción en lote (apagar todos, actualizar lecturas en lote).
3. Se Ejecuta deshacer — todos los dispositivos vuelven a su estado previo.


```

[DeviceCommandBus] execute(BatchCommand) → 3 commands queued
[BatchCommand] execute() → iterating commands in order (FIFO)
[1/3] TurnOffCommand → deviceId: 'D1-Panel-A'
→ Supabase UPDATE status='OFFLINE' D1-Panel-A ✓
[2/3] TurnOffCommand → deviceId: 'D2-Turbine-1'
→ Supabase UPDATE status='OFFLINE' D2-Turbine-1 ✓
[3/3] UpdateReadingCommand → deviceId: 'D3-Battery-B1' value: 0.0 kWh
→ Supabase UPDATE energyKwh=0.0 D3-Battery-B1 ✓
[BatchCommand] all 3 commands executed successfully
[DeviceCommandBus] BatchCommand pushed to undoStack

[DeviceCommandBus] undo() → popping BatchCommand
[BatchCommand] undo() → iterating commands in REVERSE order (LIFO)
[3/3] UpdateReadingCommand.undo() → restoring D3-Battery-B1 energyKwh: 82.4
→ Supabase UPDATE energyKwh=82.4 D3-Battery-B1 ✓
[2/3] TurnOffCommand.undo() → restoring D2-Turbine-1 status: ONLINE
→ Supabase UPDATE status='ONLINE' D2-Turbine-1 ✓
[1/3] TurnOffCommand.undo() → restoring D1-Panel-A status: ONLINE
→ Supabase UPDATE status='ONLINE' D1-Panel-A ✓
[BatchCommand] undo complete — 3 devices restored

```

Evidencia 7 — State: ciclo de vida completo de una orden

Ruta: /trading

Pasos:

1. Se Crea una nueva orden de venta usando el formulario "Crear orden — TradeOrderBuilder" (estado inicial: DRAFT).
2. Se Hace clic en "**Construir y publicar**" — la orden pasa a OPEN y aparece en el libro de órdenes.
3. Espera a que expire (ajusta el campo "Expira en (min)" a 1 minuto) o cancelála manualmente — la orden pasa a EXPIRED o CANCELLED.

```

[OrderStateMachine] new OrderContext('ORD-S-201') → state: DraftState
order: { type:'SELL', amountKwh:10.0, pricePerKwh:0.1180, source:'SOLAR', status:'DRAFT' }

[OrderContext] publish() called → delegating to DraftState
[DraftState] publish(ctx) → transition DRAFT → OPEN
→ ctx.setState(new OpenState())
→ history.push({ from:'DRAFT', to:'OPEN', at: '2026-04-27T14:33:01.000Z' })
[DraftState] transition complete | new state: OpenState

[OrderContext] publish() called again → delegating to OpenState
[OpenState] publish(ctx) → INVALID transition from OPEN
throw OrderStateError: 'Cannot publish() from state OPEN. Valid actions: match(), cancel(), expire()'

[OrderContext] expire() called → delegating to OpenState
[OpenState] expire(ctx) → transition OPEN → EXPIRED
→ ctx.setState(new ExpiredState())
→ history.push({ from:'OPEN', to:'EXPIRED', at: '2026-04-27T14:34:03.000Z', reason:'TTL_60min' })
[OpenState] transition complete | new state: ExpiredState (FINAL)

[OrderContext] cancel() called → delegating to ExpiredState
[ExpiredState] cancel(ctx) → FINAL STATE — all actions blocked
throw OrderStateError: 'Order ORD-S-201 is in final state EXPIRED. No further transitions allowed.'

[OrderStateMachine] history: DRAFT(14:33:01) → OPEN(14:33:01) → EXPIRED(14:34:03)

```

Evidencia 8 — Template Method: generación de facturas con impuestos distintos por rol

Ruta: /trading → cruce de orden exitoso

Pasos:

1. Se Crea una orden de compra y una de venta que puedan cruzarse (mismo precio, misma fuente, cantidades compatibles).
2. Se Ejecuta el matching — se genera un MatchResult.
3. Se Revisa la notificación de "Orden ejecutada" en el feed InApp o en la sección de historial.

```

[TradingMediator] processMatch() → 1 match result received
match: { buyOrderId:'ORD-B-441', sellOrderId:'ORD-S-201', matchedKwh:10.0, pricePerKwh:0.1180 }

[BaseInvoiceProcessor] processInvoice() – TEMPLATE METHOD skeleton running
Step 1: validateMatch(match) → ok
Step 2: calculateBaseAmount() → baseAmount: $1.1800 (10.0 kWh × $0.1180)

→ processorType: CONSUMER (ConsumerInvoiceProcessor)
Step 3: applyTaxes(1.18) [ABSTRACT – ConsumerInvoiceProcessor]
networkTax: 5% → $0.0590
VAT: 16% → $0.1888
taxAmount: $0.2478
Step 4: applyDiscountsOrIncentives() [HOOK – default: $0.0000]
Step 5: calculatePlatformFee(1.18) [ABSTRACT – ConsumerInvoiceProcessor]
rate: 1.5% → platformFee: $0.0177
Step 6: generateInvoiceNumber() → INV-2026-0441
Step 7: totalDueUsd: $1.4455 (base $1.18 + tax $0.2478 + fee $0.0177)

→ processorType: PRODUCER (ProducerInvoiceProcessor)
Step 3: applyTaxes(1.18) [ABSTRACT – ProducerInvoiceProcessor]
generationTax: 7% → $0.0826
taxAmount: $0.0826
Step 4: applyDiscountsOrIncentives() [HOOK – OVERRIDDEN: green incentive -2%]
discountAmount: -$0.0236
Step 5: calculatePlatformFee() → rate: 0.5% → platformFee: $0.0059
Step 7: totalDueUsd: $1.2449 (base $1.18 + tax $0.0826 - discount $0.0236 + fee $0.0059)

[TradingMediator] invoices generated → CONSUMER: $1.4455 | PRODUCER: $1.2449

```

Evidencia 9 — Chain of Responsibility: alerta bloqueada por un handler

Ruta: /iot → /notifications

Pasos:

1. Se Pon un dispositivo en modo mantenimiento (si la UI lo permite) o fuerza una lectura anómala dentro del rango normal.
2. Se Intenta disparar una alerta sobre ese dispositivo.
3. Se Revisa el feed de notificaciones.

```

[AlertValidationChain] Chain: Maintenance → Duplicate → Threshold → StaleDevice
—— CASO 1: dispositivo en mantenimiento ——
[AlertValidationChain] validate({ deviceId:'D1-Panel-A', value:0.0, inMaintenance:true })
[MaintenanceHandler] check() → inMaintenance: true
→ this.stop(log, 'Dispositivo D1-Panel-A en mantenimiento – alerta suprimida.')
→ CHAIN STOPPED at MaintenanceHandler
result: { shouldAlert: false, log: ['MaintenanceHandler: STOP'] }
—— CASO 2: alerta duplicada ——
[AlertValidationChain] validate({ deviceId:'D2-Turbine-1', value:45.2, inMaintenance:false })
[MaintenanceHandler] check() → not in maintenance → PASS, next
[DuplicateAlertHandler] check() → activeAlerts.has('D2-Turbine-1'): true
→ this.stop(log, 'Alerta duplicada para D2-Turbine-1 – ya existe una activa.')
→ CHAIN STOPPED at DuplicateAlertHandler
result: { shouldAlert: false, log: ['MaintenanceHandler: PASS', 'DuplicateAlertHandler: STOP'] }
—— CASO 3: alerta válida ——
[AlertValidationChain] validate({ deviceId:'D3-Battery-B1', value:98.5, inMaintenance:false })
[MaintenanceHandler] PASS → next
[DuplicateAlertHandler] PASS → not in activeAlerts → next
[ThresholdHandler] check() → value 98.5 > normalRange.max 95.0 → PASS (alert needed) → next
[StaleDeviceHandler] check() → lastSeen: 8s ago → device active → PASS
→ end of chain reached → shouldAlert: true
result: { shouldAlert: true, log:
['Maintenance:PASS','Duplicate:PASS','Threshold:PASS','StaleDevice:PASS'] }
[AlertValidationChain] markAlertActive('D3-Battery-B1')

```

Evidencia 10 — Memento: snapshot y restauración del libro de órdenes

Ruta: /trading

Pasos:

1. Se Con el libro de órdenes con al menos 2 órdenes activas, guarda un snapshot (busca el botón de snapshot/guardar estado si está expuesto en la UI, o ejecuta `OrderBookHistory.takeSnapshot()` desde consola del navegador).
2. Se Agrega una nueva orden o cancela una existente — el libro cambia.
3. Se Ejecuta `undo()` sobre el `OrderBookHistory` — el libro vuelve al estado guardado

```

[OrderBook] Initialized | orders: 3 | marketPrice: $0.1242 | isLocked: false

[OrderBookHistory] takeSnapshot('antes-de-limpiar')
[OrderBook] save('antes-de-limpiar')
→ new OrderBookMemento({ orders: [ORD-S-201, ORD-B-441, ORD-B-438], marketPrice:0.1242, isLocked:false
})
→ Object.freeze([...orders]) – deep copy, immutable
[OrderBookHistory] snapshot stored | count: 1/5 | label: 'antes-de-limpiar' | at: 14:35:00

[OrderBook] clearAllOrders() → 3 orders removed
[OrderBook] setMarketPrice(0.1890) → marketPrice updated
current state: orders: 0 | marketPrice: $0.1890 | isLocked: false

[OrderBookHistory] undo()
→ popping latest snapshot: 'antes-de-limpiar' (14:35:00)
[OrderBook] restore(memento)
→ this._orders = [...memento._orders] → [ORD-S-201, ORD-B-441, ORD-B-438]
→ this._marketPrice = memento._marketPrice → $0.1242
→ this._isLocked = memento._isLocked → false
[OrderBook] state restored | orders: 3 | marketPrice: $0.1242

[OrderBookHistory] undo() called again – no more snapshots
→ false (nothing to undo)

[OrderBookHistory] listSnapshots()
snapshots: [] (consumed by undo) | maxSnapshots: 5

```

10 CONSULTAS DEL PROYECTO

1. Se conecta a cualquier proveedor de precios sin tocar el sistema existente

El mercado energetico es fragmentado. Cada pais, cada operador, cada bolsa tiene su propio formato, moneda y protocolo.

EnergyTrade resuelve esto con el patron Adapter: cada API externa (OMIE en euros por MWh, ENTSO-E europeo, Octopus UK) se envuelve en una clase adaptadora que traduce el formato nativo a la interfaz interna IPriceEstimator.

El sistema nunca llama a las APIs directamente. Siempre habla con IPriceEstimator. Esto significa que agregar un nuevo proveedor (NORDPOOL, Red Electrica, cualquier bolsa regional) requiere escribir una sola clase nueva sin modificar ninguna linea del codigo existente.

Encima del adaptador, el patron Decorator apila capas de comportamiento independientes: cache TTL para no llamar la API 60 veces por minuto, logging de cada consulta para auditoria, alertas automaticas si el precio sube mas del umbral configurado, y reintentos con backoff exponencial si la red falla.

Estas capas son intercambiables. Se pueden activar o desactivar en configuracion sin tocar el adaptador ni la logica de negocio. Ningun sistema tradicional ofrece esta flexibilidad sin introducir deuda tecnica.

2. El algoritmo de matching de ordenes se cambia en vivo sin detener el sistema

La logica de como se emparejan compradores y vendedores no es fija. Depende del contexto del mercado, de la hora del dia y de la politica del operador.

EnergyTrade implementa cuatro estrategias de matching intercambiables en tiempo real: PriceFirstStrategy empareja priorizando el mejor precio (comprador dispuesto a pagar mas con vendedor mas barato), GreenFirstStrategy prioriza los cruces de energia verde certificada antes que el precio, TimeFirstStrategy respeta el orden cronologico de llegada de las ordenes (FIFO), y BestValueStrategy maximiza el volumen total emparejado por sesion.

El contexto OrderMatcher recibe la estrategia activa como parametro y la delega sin conocer su logica interna. Un operador puede cambiar de PriceFirst a GreenFirst en mitad de una sesion de trading con una sola llamada a setStrategy(). Las ordenes existentes se reemparejean inmediatamente con la nueva logica. En sistemas convencionales, cambiar el algoritmo de matching implica un despliegue, un reinicio del servicio y potencialmente una ventana de mantenimiento. En EnergyTrade es una operacion en runtime que no interrumpe ninguna operacion activa. Además, el metodo compareStrategies() permite ejecutar todas las estrategias en paralelo sobre el mismo conjunto de ordenes y comparar los resultados antes de decidir cual activar, funcionando como un simulador integrado.

3. Soporte nativo para trading de energia verde certificada con incentivos fiscales automaticos

La transicion energetica exige plataformas que no solo soporten energia verde sino que la incentiven activamente.

En EnergyTrade, el Builder de ordenes expone el metodo requireGreenCertified() que marca la orden como verde. GreenFirstStrategy prioriza automaticamente los cruces SOLAR x SOLAR y WIND x WIND sobre cualquier cruce que involucre GRID o BATTERY convencional.

Cuando se ejecuta un cruce verde, el Template Method de facturacion entra en accion. ProducerInvoiceProcessor sobrescribe el hook applyDiscountsOrIncentives() para aplicar automaticamente un descuento del 2% sobre la comision de plataforma, ademas de calcular el impuesto de generacion reducido para productores de energia limpia.

Esto no es una configuracion manual ni un campo booleano que alguien tiene que recordar activar. Es arquitectura: el flujo fiscal correcto se ejecuta automaticamente en funcion del tipo de orden y el rol del participante.

Plataformas convencionales de trading energetico no distinguen la procedencia de la energia en su logica de matching ni en su facturacion. EnergyTrade trata la certification verde como un ciudadano de primera clase del sistema.

4. Control de acceso por roles integrado en cada operacion, no como capa externa

La mayoría de los sistemas controlan el acceso en la capa de autenticacion. EnergyTrade lo controla en cada operacion individual.

El patron Proxy Protection intercepta cada llamada al servicio de dispositivos IoT antes de que llegue a Supabase. VIEWER solo puede ejecutar getDevices(). OPERATOR puede ademas addDevice() y updateReading(). Solo ADMIN puede deleteDevice(). No existe ninguna ruta en el codigo que permita saltarse esta validacion.

El mismo Proxy incorpora caching: getDevices() almacena el resultado 30 segundos. Si en ese intervalo se agrega o borra un dispositivo, el cache se invalida automaticamente. Esto reduce la carga sobre la base de datos sin sacrificar consistencia.

Cada operacion queda registrada en el audit log interno del Proxy con quien la ejecuto, cuando, cuanto tardo y si tuvo exito o fallo. Este registro no es opcional ni configurable: es parte de la arquitectura.

En sistemas donde el control de acceso es una capa externa (middleware, guards de ruta), un error de configuracion puede exponer operaciones criticas. En EnergyTrade, la seguridad esta dentro del objeto que hace el trabajo.

5. Toda accion sobre dispositivos IoT es reversible con historial completo

En redes IoT industriales, un comando ejecutado por error puede dejar sensores offline o corromper lecturas. La mayoría de sistemas no tienen mecanismo de reversion.

EnergyTrade implementa el patron Command: cada operacion sobre un dispositivo (encender, apagar, actualizar lectura, resetear) se encapsula en un objeto que sabe ejecutarse y deshacerse. DeviceCommandBus mantiene una pila de deshacer y una pila de rehacer.

TurnOffCommand guarda el estado anterior del dispositivo antes de ejecutar. Su metodo undo() restaura ese estado exacto. TurnOnCommand hace lo mismo. UpdateReadingCommand guarda el valor previo. ResetCommand guarda un snapshot completo del dispositivo.

BatchCommand envuelve multiples comandos como una operacion atomica. Si se apagan tres sensores en lote, un solo undo() los restaura todos en orden inverso (LIFO). Esto es critico en operaciones de mantenimiento donde se necesita revertir un batch completo rapidamente.

El historial de comandos es auditable: cada entrada registra que comando se ejecuto, sobre que dispositivo, con que resultado, y si fue deshecho o rehecho posteriormente. Esto cumple requisitos de trazabilidad operativa que las plataformas IoT convencionales no ofrecen.

6. Las alertas no generan ruido: filtrado inteligente en cadena antes de notificar

El mayor problema operativo de los sistemas de monitoreo IoT no es que fallen en detectar eventos, es que generan demasiado ruido. El operador aprende a ignorar las alertas.

EnergyTrade resuelve esto con AlertValidationChain, una cadena de cuatro handlers que toda alerta debe atravesar antes de ser emitida.

MaintenanceHandler la detiene si el dispositivo esta en mantenimiento programado. DuplicateAlertHandler la detiene si ya existe una alerta activa para ese dispositivo. ThresholdHandler la detiene si el valor esta dentro del rango

normal. StaleDeviceHandler la detiene si el dispositivo lleva mas de N segundos sin reportar (puede estar offline de forma esperada).

Si la alerta no pasa alguno de estos filtros, la cadena se corta en ese punto. No llega al canal de notificacion. No molesta al operador. El sistema registra que handler la bloqueo y por que.

Solo las alertas que superan los cuatro filtros se entregan. Eso significa que cuando llega una alerta, es real. El operador aprende a confiar en el sistema en lugar de ignorarlo.

Agregar una nueva regla de filtrado es anadir un handler a la cadena. No se modifica ninguna logica existente. Los handlers son independientes y reutilizables.

7. Facturacion diferenciada por rol sin duplicar ni condicionar la logica de negocio

Los participantes del mercado energetico no pagan igual. Un consumidor final, un productor solar y un operador de almacenamiento tienen reglas fiscales completamente distintas.

EnergyTrade modela esto con el patron Template Method. La clase abstracta BaseInvoiceProcessor define el esqueleto inmutable del proceso de facturacion: validar el cruce, calcular el importe base, aplicar impuestos, aplicar descuentos o incentivos, calcular la comision de plataforma, generar el numero de folio, calcular el total y generar notas.

Los pasos variables (impuestos y comision) son metodos abstractos que cada subclase implementa con su propia logica: ConsumerInvoiceProcessor aplica IVA 16% + peaje de red 5% + comision 1.5%. ProducerInvoiceProcessor aplica impuesto de generacion 7% con descuento verde opcional y comision 0.5%. StorageOperatorInvoiceProcessor tiene su propio esquema de cargos por capacidad.

Los hooks (descuentos e incentivos) tienen implementacion por defecto en la clase base pero pueden sobrescribirse. ProducerInvoiceProcessor los sobrescribe para aplicar incentivos de energia verde certificada automaticamente.

Agregar un nuevo rol de participante (por ejemplo, un agregador de demanda) es crear una nueva subclase. El flujo de facturacion principal no se toca. Los roles existentes no se modifican.

8. El libro de ordenes se puede restaurar a cualquier punto anterior sin perder trazabilidad

Los operadores cometen errores. Limpiar el libro de ordenes por accidente, cambiar el precio de referencia con el valor equivocado, o ejecutar una operacion en el momento incorrecto son situaciones reales.

EnergyTrade implementa el patron Memento sobre el OrderBook. Antes de cualquier operacion destructiva, el sistema puede tomar un snapshot del estado completo: todas las ordenes activas, el precio de mercado de referencia y el estado de bloqueo del libro.

El snapshot es inmutable. OrderBookMemento almacena una copia profunda (deep copy con Object.freeze) de las ordenes para que ninguna modificacion posterior pueda afectarlo. Solo el propio OrderBook puede leer el estado interno del Memento, preservando el encapsulamiento.

OrderBookHistory actua como Caretaker: mantiene hasta 5 snapshots en una cola FIFO. takeSnapshot() guarda el estado actual con una etiqueta descriptiva. undo() restaura el ultimo snapshot. restoreAt(i) permite volver a un punto especifico del historial. listSnapshots() muestra todos los puntos de restauracion disponibles con sus etiquetas y timestamps.

Esto convierte errores operativos en eventos reversibles en lugar de incidentes. El operador tiene una red de seguridad real, no solo un boton de confirmacion.

9. Red IoT de cualquier tamano sin degradacion de rendimiento en memoria

Escalar una red de sensores IoT tiene un costo oculto: la memoria. Cada lectura que llega al sistema puede duplicar metadatos que ya existen en miles de objetos anteriores.

EnergyTrade resuelve esto con el patron Flyweight. Una red de 50 dispositivos genera aproximadamente 3000 lecturas por minuto. Sin Flyweight, cada objeto IoTReading almacenaria sus propios campos de tipo de sensor: unidad de medida, etiqueta de display, color de visualizacion, rango normal, formato de salida. Son 5 campos duplicados en cada una de las 3000 lecturas por minuto.

Con Flyweight, esos 5 campos se almacenan una sola vez en SensorTypeFlyweight, una instancia por tipo de sensor. Las 10000 lecturas del

tipo POWER comparten la misma instancia. El ahorro de memoria es aproximadamente del 97% en el estado intrínseco de las lecturas.

Para la estructura de la red, el patrón Composite permite tratar dispositivos individuales y grupos de dispositivos con la misma interfaz `IEnergyNode`. Una llamada a `portfolio.getTotalKwh()` recorre recursivamente toda la jerarquía (Portfolio -> Granja Solar -> Panel A, Panel B, Panel C) sin que el código cliente sepa si está hablando con un nodo hoja o un grupo.

Agregar un nuevo nivel de agrupación (región, planta, edificio) es añadir un nodo Composite. El código cliente no cambia.

10. Una sola llamada desde la interfaz orquesta todo el backend de forma transparente

La complejidad interna de una plataforma de trading no debería filtrarse hacia los componentes de interfaz. Cada página que necesita conocer cinco subsistemas distintos es deuda técnica esperando fallar.

`EnergyTradingFacade` expone cinco operaciones de alto nivel que internamente coordinan todos los subsistemas: `createAndPublishOrder()` coordina Builder, Supabase y Bridge de notificaciones. `cloneTemplateAndPublish()` coordina Prototype Registry, Supabase e InApp Channel. `getMarketSummary()` coordina tres Adapters externos, el pipeline Decorator y `PriceAlertNotifier`. `getPortfolioStatus()` consulta órdenes activas y dispositivos en Supabase. `getAvailableTemplates()` accede al `OrderTemplateRegistry`.

El componente React de la página de Trading llama a una sola función y recibe el resultado. No conoce Builder, no conoce Supabase, no conoce Bridge, no conoce Observer. Si mañana se cambia la implementación interna de cualquier subsistema, la página no se entera.

Para operaciones que involucran múltiples subsistemas en coordinación (como procesar un cruce de orden que involucra matching, facturación, eventos y alertas), `TradingMediator` centraliza la coordinación. Cada subsistema solo conoce al Mediator. Las dependencias directas entre subsistemas son cero.

`MarketEventBus` completa el cuadro: los cambios de estado se propagan a todos los observadores interesados sin que el emisor conozca a los receptores. La UI se actualiza en tiempo real sin polling. El sistema reacciona a los eventos, no los busca.

LINK DE EJECUCION DEL PROYECTO

Damos clic en el link

<https://wfw3qsm4-3000.use.devtnnls.ms/>

iniciamos sesión en GitHub y listo 😊

En el redime del proyecto en GitHub también esta el link